



**UNIVERSITY OF
SIALKOT**
A CHARTERED UNIVERSITY

**Faculty of Computing & IT
University of Sialkot**

“Digital Electoral System”

**BS Information Technology
Spring 2023-Fall 2025**

Supervisor
Intakhab Alam

Co-Supervisor
Dr. Jabar Mehmood

Submitted By

Adil Saleem 23201003-006

Umar Iftikhar 23201003-016

Faculty of Computing & IT
University of Sialkot

PROJECT ID	1
-------------------	----------

Number of Group members	2
--------------------------------	----------

TITLE	Digital Electoral System
--------------	--------------------------

SUPERVISOR NAME	Intakhab Alam
CO SUPERVISOR NAME	Dr. Jabar Mahmood

Sr. No.	Name	Roll Number	Email Address
1	Adil Saleem	23201003-006	1230200027@uskt.edu.pk
2	Umar Iftikhar	23201003-016	12302000095@uskt.edu.pk
3			
4			

CHECKLIST

Number of pages in this report

107

We have enclosed the soft-copy of this document along-with the codes and scripts created by myself/ourselves

YES / NO

My/Our supervisor and co supervisor has attested the attached document

YES / NO

I/We confirm to state that this project is free from any type of plagiarism and misuse of copyrighted material

YES / NO

Members Signatures

STATEMENT OF SUBMISSION

This is certified that **Adil Saleem** and **Umar Iftikhar** Roll No. **23201003-006 & 23201003-016** have successfully completed the final year project named as Digital Electoral System at the Department of Information Technology, University of Sialkot, to fulfill the requirement of the degree of BS in Information Technology.

Project Supervisor

Project Management Office
Faculty of CS&IT -USKT

Head of the Department

ACKNOWLEDGEMENT

We truly acknowledge the cooperation and help make by **Dr Jabar Mahmood and Intakhab Alam**. They have been a constant source of guidance throughout the course of this project. We are also thankful to our friends and family whose silent support led us to complete our project.

Date:

DECLARATION

Muhammad Adil S/O Muhammad Saleem 23201003-006 and **Umar Iftikhar S/O Malik Iftikhar 23201003-016** are students of Bachelor of Information Technology, Department of Computing & Information Technology, University of Sialkot, Pakistan, hereby solemnly declare that the data quoted in the report titled “**Digital Electoral System**” is based on our original work, and has not yet been submitted or published elsewhere.

Adil Saleem

23201003-006

16-02-2024

Umar Iftikhar

23201003-016

16-02-2024

CERTIFICATIONS

1. I certify that the project titled “**Digital Electoral System**” is under my supervision with students **Adil Saleem Roll # 23201003-006** and **Umar Iftikhar Roll # 23201003-016** of Computing & Information Technology, University of Sialkot, Pakistan.

Supervisor Name: Intakhab Alam

Department of Information Technology

Faculty of Computing & Information Technology

University of Sialkot, Punjab, Pakistan.

Email: intakhab.alam@uskt.edu.pk

Date: 16-02-2024

2. I certify that the project titled “**Digital Electoral System**” is under my supervision with students **Adil Saleem Roll # 23201003-006** and **Umar Iftikhar Roll # 23201003-016** of Computing & Information Technology, University of Sialkot, Pakistan.

Co-Supervisor Name: Dr. Jabar Mehmood

Department of Information Technology

Faculty of Computing & Information Technology

University of Sialkot, Punjab, Pakistan.

Email: jabarmehmood@outlook.com

Date: 16-02-2024

Table of Content

CHAPTER-1 INTRODUCTION	1
1.1 Introduction	1
1.2 Existing System.....	1
1.3 Objectives.....	1
1.4 Project Motivation.....	2
1.5 Project / Product Feasibility Report	2
1.5.1 Technical Feasibility	2
1.5.2 Operational Feasibility	3
1.5.3 Specification Feasibility	3
1.5.4 Motivational Feasibility	3
1.5.5 Schedule Feasibility	3
1.5.5.1 Gantt Chart.....	3
1.5.6 Information Feasibility	4
1.5.7 Legal & Ethical Feasibility.....	4
1.5.8 Project Scope.....	4
1.6 Task Dependency Table	4
1.7 CPM – Critical Path Method.....	4
1.7.1 Specify the Individual Activities.....	4
1.7.2 Determine the Sequence of the Activities	5
1.7.3 Draw the Network Analysis Diagram	5
1.8 Introduction to Team Members & their Skills	6
1.9 Task and Member Assignment Table	6
1.10 Tools and Technology	6
1.11 Vision Document	7
1.12 Risk List	7
1.13 Product Features.....	8
CHAPTER-2 LITRERATURE REVIEW& SYSTEM SPECIFICATION.....	9
2.1 Literature Review.....	9
2.1.1 Biometric Voting Systems	10
2.1.2 IoT-Enabled Voting Systems	10
2.1.3 Challenges in Traditional Voting	10
2.2 System Specification	11
2.2.1 Functional Requirements.....	11
2.2.2 Non-Functional Requirements	11
2.3 System Architecture	11
2.3.1 Hardware Requirements	11
2.3.2 Software Requirements	12
2.4 Comparative Analysis	12
2.5 Scope of the System	12
2.6 Main Modules	12
2.6.1 Arduino Uno R3.....	13
2.6.2 R305 Fingerprint Sensor Module.....	14
2.6.3 Lcd Display (24x4)	14
2.6.4 Arduino Serial Thermal Printer.....	15
2.6.5 Jumper Wire	15
2.6.6 Bread Board	16
2.6.7 Buzzer	16

2.6.8 Push Button	17
2.6.9 Battery Management System (BMS)	17
2.6.10 Wi-Fi ESP32 Microcontroller Module.....	18
2.6.11 18650 Lithium Ions Cell	18
2.6.12 12V Power Adapter.....	19
CHAPTER-3 DESIGN & METHODOLOGY	20
3.1 System Design Overview	20
3.2 Architectural Design	20
3.2.1 Hardware Architecture	21
3.2.2 Software Architecture.....	21
3.2.3 System Workflow.....	21
3.3 Purposed Methodology	22
3.3.1 Agile Development Process	23
3.3.2 Feedback Cycles	23
3.3.3 Development Phase.....	23
3.3.4 Visual Aids.....	23
3.4 Component Design.....	24
3.4.1 Hardware Components	24
3.4.2 Software Components	24
3.5 Security Measures	24
3.6 Challenges and Solutions	25
3.7. Terminology	25
3.8 Working of Components	26
3.9 Working of Project.....	26
3.10 Block Diagram	27
3.11 Use Case Diagram.....	27
3.12 Architecture Diagram.....	28
3.13 Circuit Diagram.....	28
3.15 Sequence Diagram	29
3.16 Class Diagram	29
3.17 Flow Chart Fingerprint Enrollment.....	30
3.18 Flow Chart Vote Casting.....	30
CHAPTER-4 RESULTS AND TESTING	31
4.1 System Overview.....	31
4.1.1 System Features	31
4.2 System States	31
4.3 Testing Methodology	37
4.3.1 Test Environment	37
4.3.2 Test Phases.....	38
4.4 Testing and Results	38
4.4.1 Login Test Case Description: To test the functionality of login.	38
4.4.2 Register Test Case Description: To test the functionality of Register.	39
4.4.2 User Dashboard Test Case Description: To test the functionality of the User Dashboard.	39
4.5 Test Results	40
4.6 Firebase Data Management.....	41
4.6.1 Data Storage and Retrieval in Firestore	41
4.6.2 Firebase Realtime Database	41
4.7 Key Observations	41

CHAPTER-5 CONCLUSION AND FUTURE WORK.....	42
5.1 Conclusion.....	42
5.2 Limitations	42
5.3 Future Work	42
5.4 Final Remarks	43
APPENDIX A	45
CODE ARDUINO IDE.....	45
MAIN FILE.....	45
HEADER FILE 1	54
HEADER FILE 2	55
ESP32 CODE	55
RESULT CODE.....	57
UPLOAD RESULT CODE.....	58
UPLOAD RESULT ADMIN CODE	59
Application Login Activity	61
XML Code	61
Java Code	64
Application Register Activity	68
XML Code	68
Java Code	72
Application User Activity	75
XML Code	75
Java Code	77
Application Admin Activity	86
XML Code	86
Java Code	90
APPENDIX B	93
Project Timeline	93
APPENDIX C	94
SUPERVISOR MEETING LOG	94

LIST OF FIGURES

Figure 1.1 Schedule Feasibility Gantt Chart	3
Figure 1.2 Network Analysis Diagram	5
Figure 1.3 Product / Project Features & Decomposition.....	8
Figure 2.1 Arduino Uno R3	13
Figure 2.2 Fingerprint Sensor R305.....	14
Figure 2.3 LCD Display (24x4)	14
Figure 2.4 Serial Thermal Printer.....	15
Figure 2.5 Jumper Wire.....	15
Figure 2.6 Breadboard.....	16
Figure 2.7 Buzzer	16
Figure 2.8 Push Button.....	17
Figure 2.9 Battery Management System.....	17
Figure 2.10 Wi-Fi Microcontroller	18
Figure 2.11 18650 Lithium Ion Cell.....	18
Figure 2.12 12V Power Adapter	19
Figure 3.1 Purposed Methodology	22
Figure 3.2 Block Diagram.....	27
Figure 3.3 Use Case Diagram	27
Figure 3.4 Architecture Diagram	28
Figure 3.5 Circuit Diagram	28
Figure 3.6 Sequence Diagram	29
Figure 3.7 Class Diagram.....	29
Figure 3.8 Flow Chart Fingerprint Enrollment	30
Figure 3.9 Flow Chart Vote Casting	30

Figure 4.1 Idle State	31
Figure 4.2 Voter Verification State	32
Figure 4.3 Voting State 1	32
Figure 4.4 Voting State 2	32
Figure 4.5 Voting State 3	33
Figure 4.6 Voting State 4	33
Figure 4.7 Result Compilation State 1	33
Figure 4.8 Result Compilation State 2	33
Figure 4.9 Result Compilation State 3	33
Figure 4.10 Result Compilation State 4	33
Figure 4.11 Login Page	34
Figure 4.12 Register Page	34
Figure 4.13 User Dashboard.....	35
Figure 4.14 Admin Dashboard	36
Figure 4.15 Data Synchronization with Firebase	37

LIST OF Tables

Table 1.1 Task Dependency	4
Table 1.2 Sequence of Activities.....	5
Table 1.3 Team Members & Skills	6
Table 1.4 Task & Member Assignment	6
Table 1.5 Hardware Risk List.....	7
Table 1.5 Software Risk List.....	7
Table 1.6 Comparative Analysis	12
Table 1.7 Login Test Case.....	38
Table 1.8 Register Test Case.....	39
Table 1.9 User Dashboard Test Case	39
Table 2.0 Test Result.....	40

ABSTRACT

Conducting free and fair elections in the world's 7th largest democracy has always been challenging for the election commission, requiring significant financial resources to ensure riot-free voting. However, electoral mal-practices such as rigging have become more common, potentially distorting the people's true verdict. We proposed an electronic voting system utilizing biometric technology to enhance accuracy, speed, and integrity. By using unique thumb impressions for voter identification, our proposed system offers a significant improvement over current method. Before elections, a database of all eligible voters' thumb impressions is established. On election day, voters' thumb impressions are matched with the database records to grant or deny voting access. This proposed method effectively prevents multiple voting and fraudulent entries. All voting machines are capable to generate real-time result compilation. The following result is then uploaded to the Application authorized by concerned organization. This proposed system not only ensures more accurate and timely results but also reduces the overall costs and maintenance expenses associated with conducting elections.

CHAPTER-1 INTRODUCTION

1.1 Introduction

Ensuring the accuracy, security, and efficiency of the voting process is critical in the context of democratic elections. Although fundamental to voting history, traditional ballot paper voting has faced several difficulties throughout the years, including voter fraud, protracted counting procedures, and vote tampering. By minimizing these problems, modern technology can provide a more dependable and efficient voting experience. Electronic voting systems have shown great promise in improving the speed and integrity of elections, especially those that include biometric verification.[2]

1.2 Existing System

Traditional voting systems can be complex and time-consuming. The traditional election method of paper ballot (a long, time-consuming process where votes are cast and counted by hand) requires more manual labor, energy, workforce, and time. This process has potential problems such as increased costs and voter fraud.[1] When the election process is completed, each ballot paper must be checked manually to count the votes, which takes time and effort. The physical security of paper ballots (theft, burning, inking) is also a concern. False ballots can be printed, leading to fake votes being cast. Additionally, there is the possibility of errors or mistakes made by voters, such as missing a field or putting in the wrong information, which can invalidate the ballot. There are also significant costs associated with printing, storing, and transporting the ballots. These drawbacks can result in issues such as repolling, requiring additional expenditures on the part of the government and taxpayers' money.[5]

The designed system is implemented by identifying the following problems:

- Manual vote counting is time-consuming and labor-intensive.
- Physical security concerns such as theft, destruction, and tampering of paper ballots.
- The potential for voter fraud through false ballot printing.
- Human errors by voters that can invalidate ballots.
- High costs associated with printing, storing, and transporting paper ballots.
- The need for repolling due to various issues, leading to increased governmental and taxpayer expenses.
- Limited accessibility for voters with disabilities, making it challenging for them to participate in the traditional voting process.
- Inconsistent handling and storage of ballots across different polling stations, leading to discrepancies and potential loss of votes.

1.3 Objectives

- **Enhanced Security:** Ensure that only registered and authorized voters can cast their votes through biometric verification (fingerprint or other biometric data).
- **Prevent Fraud:** Eliminate the possibility of duplicate voting and impersonation by uniquely identifying each voter with their biometric data.
- **Accuracy:** Improve the accuracy of the voting process by reducing human errors in

voter identification and vote counting.

- **Efficiency:** Speed up the voting process by automating voter verification and vote counting, reducing long queues and waiting times.
- **Real-time Data:** Enable real-time data collection and monitoring of voter turnout and voting patterns.
- **Cost-effectiveness:** Reduce the overall cost of elections by minimizing the need for extensive manpower and paper ballots.
- **Scalability:** Design the system to be scalable and adaptable to different sizes and types of elections, from local to national levels.[2]

1.4 Project Motivation

The motivation for developing a biometric voting machine is to address the challenges and limitations of traditional voting systems by enhancing security, preventing electoral fraud, improving accuracy and efficiency, and boosting voter trust. By ensuring only authorized voters can vote and automating vote counting, the system reduces human error and speeds up the voting process. It promotes accessibility and inclusivity, reduces costs, supports environmental sustainability, and enables real-time monitoring of elections. Furthermore, the scalable and adaptable nature of biometric voting machines encourages technological advancement and can be applied to various electoral contexts, ensuring a secure, efficient, and trustworthy electoral process.[3]

1.5 Project / Product Feasibility Report

IOT and automation have become the most important part in everyday life of people. It has given ease to the people around the world. Our project of electronic voting using biometrics and mobile application is feasible because it is being used in all developed countries like UK, USA and CANADA and also countries with huge Population such as INDIA, Bangladesh and Brazil.[4] But this system has not been implemented in Pakistan, so that's the motivation to work on this project, because it will be absolutely revolutionary in conducting free and fair elections in the country and eliminate existing problems like election fraud, voter fraud and also gain back people's trust in electoral process.[5]

There are many types of feasibilities:

- Technical
- Operational
- Specification
- Motivational
- Schedule
- Information
- Legal & Ethical

1.5.1 Technical Feasibility

This project is technically feasible and can be developed as it has been widely implemented and tested in major democracies and have been proven to be efficient than previous election process. This project is based on, Arduino board and other IOT tools alongside mobile

application. We have handled all the technical things through local connection and handheld device. No other specification needed to run this application.[1]

1.5.2 Operational Feasibility

In this project, we tried our best to produce such a product that users can easily operate without facing any technical difficulties. We made this voting system automated and easy to operate. This is helpful for people that comes with less educated background.[2]

1.5.3 Specification Feasibility

All requirements completed within time. All requirements are clear. We have acquired requirements for the project by keeping the fact in view the user priorities, their demands and the problems they faced. All requirements are the features that our system must have or a constraint that must be accepted for end users. Specification feasibility also included hardware and software specification required for completion of our project.[3]

1.5.4 Motivational Feasibility

Our supervisor motivates our group regular basis so that's why the members of our group are fully motivated to perform the formidable tasks of developing this system. We realize that how big this is going to be and we as a team are the source of motivation for ourselves.[4]

1.5.5 Schedule Feasibility

For timely completion of our project, we have developed a Gantt chart. Our project will be done before 2nd May, 2024 in the Gantt chart. The assessment and evaluation of the completion of our project with the available staff and resources with in time is very essential. Figure 1.1 represent Schedule Feasibility Gantt Chart.[5]

1.5.5.1 Gantt Chart

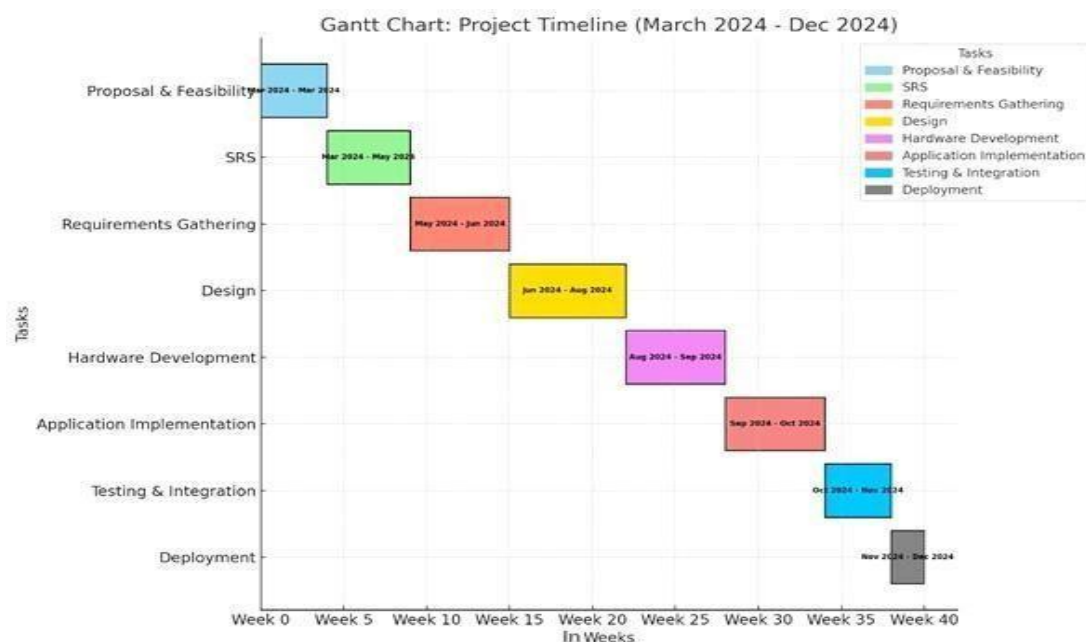


Figure 1.1 Schedule Feasibility Gantt Chart

1.5.6 Information Feasibility

All given information is completely reliable, authentic and meaningful based on a detailed and thorough research done by our group. No confusion about the requirement. Information quality is very good and true information is given. So, our project is feasible. Our project is feasible in this context because we collected all the information related to our project from authentic sources such as credible research papers.[2]

1.5.7 Legal & Ethical Feasibility

This project is exclusively being developed for the people so that they don't get worried about legal & ethical feasibilities, there are no unethical activities (hidden camera, voice recorder) in our project which can affect the voter privacy and influence on electoral process. This project is feasible legally and ethically as it does not violate ECP regulations policy of election code.[4]

1.5.8 Project Scope

This product scope ensures that the biometric voting machine project is meticulously planned, executed, and evaluated to achieve its objectives of enhancing electoral process security, efficiency, and transparency to achieve successful election in a 220 million nation.[3]

1.6 Task Dependency Table

Table 1.1 represent Task dependency table that describe all tasks and their durations.

Table 1.1 Task Dependency

Task	Duration (Weeks)	Dependencies
T1	5	None
T2	3	A (Proposal & Feasibility)
T3	5	B (System Requirement Specification)
T4	6	B, C (SRS, Requirement Gathering)
T5	8	D (Design)
T6	5	D, E (Design, Hardware Development)
T7	6	E, F (Hardware Development & App Implementation)
T8	2	G (Testing & Integration)

1.7 CPM – Critical Path Method

The CPM analysis highlights the critical tasks (**A, B, C, D, E, G, H**) and provides a roadmap to ensure timely completion of the "Digital Electoral System" project. By managing the critical path, the team can allocate resources efficiently and avoid delays, ensuring project success.

1.7.1 Specify the Individual Activities

In “Digital Electoral System”, we have divided our tasks broadly in following activities.

These activities are performed on weekly basis and their Gantt Chart is also developed alongside.

- Proposal & Feasibility
- System Requirement Specification
- Requirement Gathering
- Design
- Hardware Development
- App Implementation
- Testing & Integration
- Development

1.7.2 Determine the Sequence of the Activities

Table 1.2 represent activities along with duration that are given below.

Table 1.2 Sequence of Activities

Activity	Name (Activity)	Immediate Predecessor	Duration (Weeks)
A	Proposal & Feasibility	None	5
B	System Requirement Specification	A	3
C	Requirement Gathering	B	5
D	Design	B, C	6
E	Hardware Development	D	8
F	App Implementation	D, E	5
G	Testing & Integration	E, F	6
H	Deployment	G	2

1.7.3 Draw the Network Analysis Diagram

Figure 1.2 represents Network Analysis Diagram for the above-mentioned activities are mentioned below:

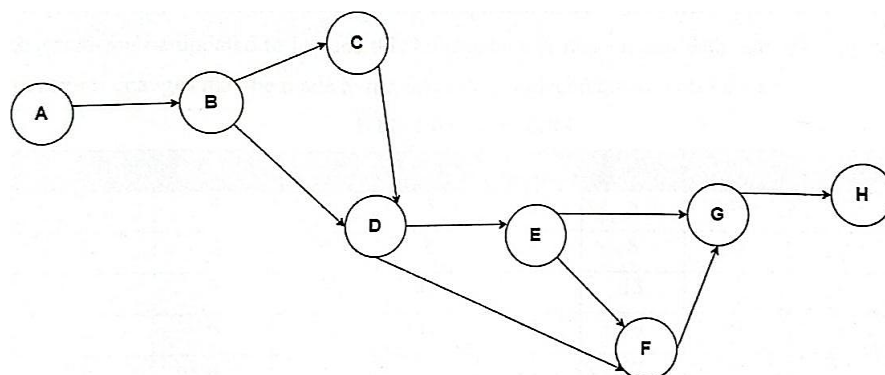


Figure 1.2 Network Analysis Diagram

1.8 Introduction to Team Members & their Skills

The name, introduction, skills and responsibilities of the team members regarding their project are given and represented by Table 1.3 as follow:

Table 1.3 Team Members & Skills

Name	Introduction	Skills	Responsibility
Adil Saleem	Student of BSIT University of Sialkot	Programmer, Developer	Hardware, Software, Documentation
Umar Iftikhar	Student of BSIT University of Sialkot	Programmer, Developer	Hardware, Software, Documentation

1.9 Task and Member Assignment Table

The Task and members assignment table according to assigned tasks is given and represented by Table 1.3 below:

Table 1.4 Task & Member Assignment

Task	Duration (Weeks)	Dependencies
T1	5	M2
T2	3	M1, M2
T3	5	M2
T4	6	M1
T5	8	M1, M2
T6	5	M1
T7	6	M2
T8	2	M1

1.10 Tools and Technology

The Tools and technologies are described as follows:

- **OS** - Windows 11 Pro.
- **IDE** -Arduino IDE is a platform that is used to code.
- **Circuit Designer** —For the practice implementation of the project.
- **Editor** - Adobe Photoshop/E-draw Max/Draw.io is used for designing.

- **IOT** — Internet-of-Things is the technology used in this project.
- **Android Studio** — To write the logics and build .apk file for android device.
- **Target Platforms** — The target platforms are ECP, government.

1.11 Vision Document

"Digital Electoral System" is an IOT based project, which once developed will assist people to use it. In our vision document the stakeholders are Users (Voters) and Management (ECP). The purpose of our project is to facilitate the fair and transparent voting process. Some of the features on which all stakeholders are agreed are listed here:

- Voter Verification
- Alert on Voter Duplicity
- Admin Authentication
- Print Votes & Results
- Total Voter Turn Out

1.12 Risk List

Table 1.5 represents the hardware risk list & software risk list below:

Table 1.5 Hardware Risk List

Risks	How To Overcome?
Voltage Compatibility	Adjust Voltage with regulators
Circuit can't fulfill over requirements	Can be filled on new board
Board can't support Wi-Fi	Adjust Wi-Fi module with accurate pins

Table 1.5 Software Risk List

Risks	How To Overcome?
Learn new language	Learn Circuit Programming
Controlling Hardware	Controlling Hardware using Arduino and Circuit Designer Software's

1.13 Product Features

The diagram illustrates the structure of a Digital Electoral System, highlighting its key users, operations, and components. It involves both voters and administrators in the process, with core operations like verification, authentication, vote counting, and printing of votes. Components such as mobile phones, fingerprint sensors, and thermal printers support the system's functionality. The goal is to ensure secure identification, accurate voting, and transparent election management using digital technology. Figure 1.3 represent the Product Features below:

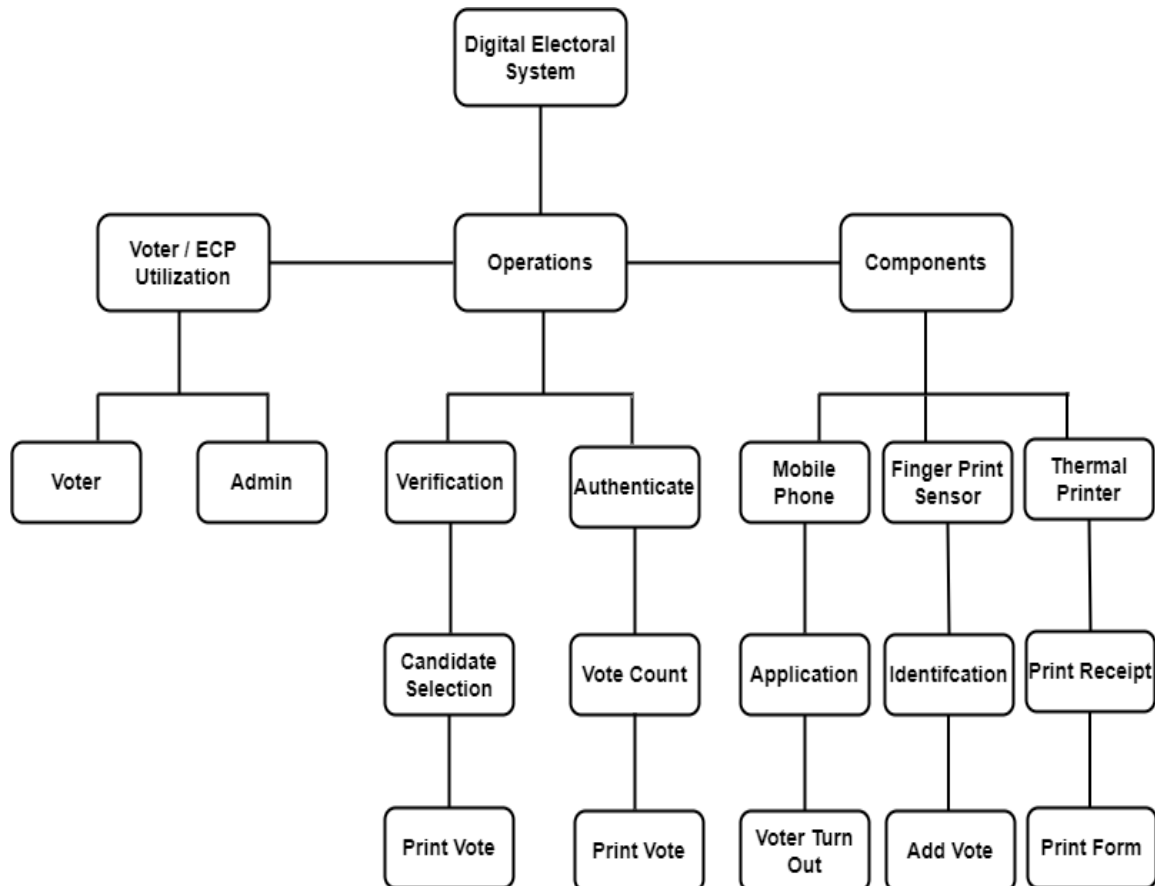


Figure 1.3 Product / Project Features

CHAPTER-2 LITRERATURE REVIEW& SYSTEM SPECIFICATION

2.1 Literature Review

The biometric electronic voting machine presents a robust alternative to conventional voting methods. By leveraging unique fingerprint patterns for voter identification, it ensures a higher level of security and accuracy. The system's straightforward design and low production cost further enhance its practicality for widespread implementation. As such, it holds the potential to significantly improve the electoral process, making it more secure, efficient, and trustworthy [1].

The AADHAR Based Electronic Voting Machine leverages the AADHAR database to streamline and secure the voting process in India. Voters use their AADHAR card and fingerprint for authentication, ensuring that only legitimate votes are cast. This system, utilizing Arduino and Raspberry Pi, addresses issues of voter fraud, time consumption, and vote tampering prevalent in traditional EVMs [2].

By automating the verification and voting process, it enhances accuracy and reliability. Additionally, it provides real-time results and voter verification, thereby promoting transparency and trust in the electoral process [3].

The Smart and Secure Voting Machine using Biometrics aims to resolve issues inherent in traditional EVMs by incorporating biometric verification and GSM technology. This system ensures that votes are accurately recorded and verified through a two-step process involving fingerprint authentication and SMS confirmation. By storing data on a cloud-based database, it provides a reliable and tamper-proof voting method. This approach not only enhances voter confidence but also ensures that each vote is counted correctly and attributed to the intended candidate, thus improving the integrity of the whole electoral process [4].

The role of information technology in the electoral process and highlights the transition from traditional paper ballots to electronic voting machines (EVMs), emphasizing the benefits of EVMs, such as faster and more accurate vote counting, reduced human error, and improved accessibility for physically challenged voters. The article discusses the initial skepticism regarding EVMs and the successful adaptation by both literate and illiterate voters. Additionally, the study addresses the potential for EVM tampering and the need for robust security measures. The authors suggest regular voter surveys, awareness campaigns, and strict verification processes to enhance the integrity of the electoral process. Despite the advantages, the article acknowledges concerns about the vulnerability of EVMs to hacking and the need for continuous improvements to ensure trust in electronic voting systems. The study concludes that EVMs are essential for modern elections, provided that their implementation follows strict security protocols [5].

Development of Electronic Voting Machine with the Inclusion of Near Field Communication ID Cards and Biometric Fingerprint Identifier proposes a secure electronic voting system using

NFC ID cards and biometric fingerprint verification to prevent voter fraud. This system addresses the issues of manual tallying errors and online security risks present in traditional voting systems. Key components include an NFC controller shield for Arduino, an optical fingerprint sensor, and a 16x2 LCD display, which work together to verify voter identity and ensure only authorized votes are cast. By eliminating live network use, the system enhances security and reliability, offering a significant advancement in electronic voting technologies [5].

The article provides a comprehensive analysis of electronic voting systems, including their evolution, technological components, and challenges. The authors emphasize the significance of electronic voting machines (EVMs) in enhancing the electoral process by addressing issues associated with traditional paper ballots. The review covers various types of EVMs, highlighting the differences in hardware, software, and security features. The integration of biometric systems in EVMs is discussed as a promising approach to improve voter authentication and prevent fraud. The paper also explores the potential of networked EVMs for remote voting, ensuring accessibility and efficiency. Additionally, the authors address concerns regarding software vulnerabilities, insider threats, and the importance of a verifiable audit trail. The article concludes with recommendations for future innovations in EVM technology to enhance security, transparency, and voter confidence [6].

The biometric electronic voting machine offers a strong alternative to traditional voting systems. Utilizing unique fingerprint patterns for voter identification, it guarantees enhanced security and accuracy. The system's simple design and low production costs make it practical for widespread use. This technology has the potential to greatly improve the electoral process by making it more secure, efficient, and reliable. Its implementation can reduce the risk of voter fraud and errors, thus fostering greater trust in the voting system. Overall, the biometric electronic voting machine stands out as a viable solution for modernizing elections and ensuring the integrity of democratic processes [7].

2.1.1 Biometric Voting Systems

Biometric systems use unique physical characteristics, such as fingerprints, for authentication. Studies have demonstrated their effectiveness in preventing voter fraud and ensuring secure access. For instance, biometric voting systems in countries like India and Estonia have shown improved accuracy and transparency in elections.

2.1.2 IoT-Enabled Voting Systems

IoT-enabled devices facilitate real-time data transmission and remote monitoring, making electoral processes more efficient. Research highlights the benefits of integrating IoT technology to streamline vote collection, result compilation, and system diagnostics.

2.1.3 Challenges in Traditional Voting

Traditional voting systems face challenges such as:

- High costs associated with paper ballots and manual labor.
- Vulnerability to tampering and fraud.

- Time-consuming vote counting processes.
- Limited accessibility for people with disabilities.

These challenges necessitate the adoption of advanced systems like the "Digital Electoral System," which addresses these limitations.

2.2 System Specification

The "Digital Electoral System" was designed to meet specific functional and non-functional requirements, ensuring security, efficiency, and usability.

2.2.1 Functional Requirements

1. Biometric Authentication:

- Verify voter identity using fingerprints.
- Prevent duplicate or unauthorized voting.

2. Voting Process:

- Allow voters to cast their votes electronically after successful authentication.
- Provide real-time feedback via an LCD display.

3. Result Compilation:

- Automate the process of tallying votes.
- Upload results securely to the election commission's portal.

4. Audit Trail:

- Print a physical receipt of the vote for transparency to match the result generated by machine in case of cross verification required.

2.2.2 Non-Functional Requirements

1. Performance:

- Handle up to 100 concurrent users without delays.

2. Security:

- Encrypt biometric and voting data.
- Implement tamper detection mechanisms.

3. Scalability:

- Adapt to elections of varying sizes, from local to national.

4. Usability:

- Provide an intuitive interface for voters and administrators.

2.3 System Architecture

The system's architecture integrates hardware and software components to ensure seamless functionality:

2.3.1 Hardware Requirements

- **Arduino Uno R3:** Controls system operations.
- **Fingerprint Sensor (R305):** Captures and authenticates voter fingerprints.
- **LCD Display (24x4):** Provides real-time feedback to voters.
- **Thermal Printer:** Prints voting receipts for transparency.

- **Wi-Fi Module:** Enables real-time result uploading.

2.3.2 Software Requirements

- **Arduino IDE:** Develops and uploads firmware to the Arduino board.
- **Android Studio:** Develops the mobile application interface.
- **Database Management System:** Stores voter data and voting records securely.

2.4 Comparative Analysis

The following table 1.6 compares traditional voting systems with the "Digital Electoral System":

Table 1.6 Comparative Analysis

Feature	Traditional Voting System	Digital Electoral System
Authentication	Manual verification	Biometric fingerprint authentication
Vote Counting	Manual, time-intensive	Automated, real-time
Fraud Prevention	Susceptible to tampering	Highly secure with encryption
Accessibility	Limited for disabled voters	Inclusive and user-friendly
Cost Effectiveness	High due to paper and labor	Reduced through automation

2.5 Scope of the System

The scope of implementing an Electronic Voting Machine (EVM) with biometric authentication encompasses several key areas that can significantly enhance the voting process:

1. Enhanced Security and Fraud Prevention:

Biometric authentication, such as fingerprint recognition, ensures that each voter is uniquely identified, eliminating the possibility of multiple votes by the same individual and reducing the chances of voter impersonation.

2. Improved Accuracy and Reliability:

The use of biometric data ensures a high level of accuracy in voter identification, thereby reducing errors associated with manual verification processes. This leads to more reliable election outcomes.

3. Streamlined Voting Process:

The integration of user-friendly interfaces, such as a 24x4 LCD display and push buttons, simplifies the voting process for voters, making it more accessible and efficient.

4. Transparency and Accountability:

The inclusion of a serial thermal printer provides physical receipts of votes, which adds a layer of transparency and allows for audit trails. This feature can help build trust in the electoral process among voters.

5. Cost-Effectiveness and Practicality:

The straightforward design and low production costs of the biometric EVM make it a practical solution for widespread implementation across various regions, including those with limited resources.

6. Portability and Power Management:

The use of a battery management system ensures that the EVM can function reliably even in areas with unstable power supplies, making it suitable for remote and rural locations.

7. Connectivity and Remote Monitoring:

The inclusion of a Wi-Fi module (ESP32 Microcontroller) to facilitate the communication between application and IOT Devices over internet.

2.6 Main Modules

There are some main modules mandatory to fulfil the operational requirements listed below:

2.6.1 Arduino Uno R3

Arduino uno R3 is an electronic prototype platform/board based on ATmega AVR. Microcontroller has ATmega 328 (8 bit,6Mhz). It operates on +5 volts. It has 14 digital I/O pins (of which 6 are PWM) (pulse width modulation). The voltage regulator converts the input to 5v. Subsequently, this signal powers up the Arduino Uno board. There is a reset button on Arduino which does pretty much same as unplugging the board and plugging it back. It restarts your program from beginning. We also use crystal oscillator to executed I synchronization with clock and provide timing for different operation into microcontroller. Figure 2.1 represents the Arduino Uno R3 in this figure it completely describes the structure

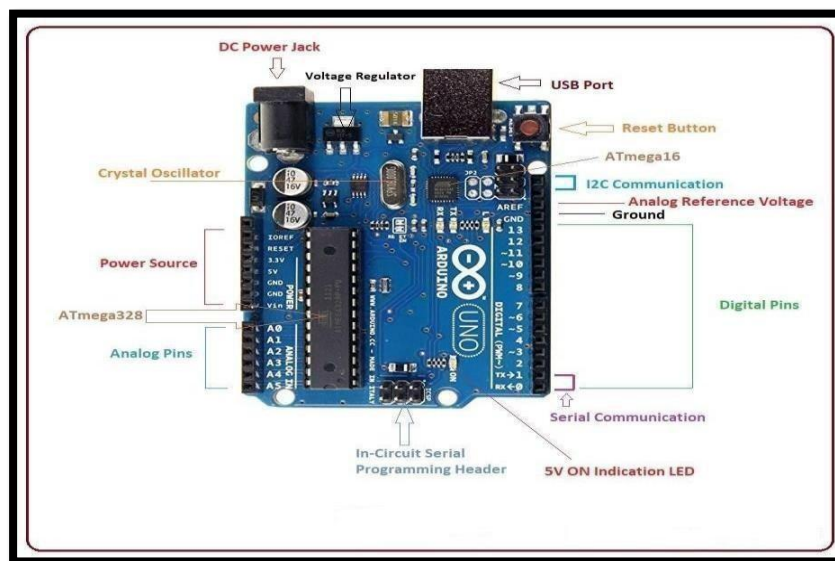


Figure 2.1 Arduino Uno R3

2.6.2 R305 Fingerprint Sensor Module

R305 fingerprint sensor is capable of capturing detailed fingerprint images. It can store up to 1,000 fingerprints. The module has digital output. Fingerprint capture and matching (acquisition time: $\leq 1s$, matching time: $\leq 1s$). Can store and manage fingerprint templates on the device. It operates on +3.6-6.0 volts. Features image processing algorithms for high security. Capable of improving fingerprint recognition over time by learning slight variations). Figure 2.2 represents the structure of R305 Fingerprint Module.



Figure 2.2 Fingerprint Sensor R305

2.6.3 Lcd Display (24x4)

There are two sections pins on the whole (24x4) LCD module. Some of them are data pins and some are command pin. Somehow, every pin has a role in controlling a single pixel on the display. Additionally, all the input/output pins of this module are shown in the pin out diagram. Figure 2.5 represent the lcd display hardware. Figure 2.3 represents structure of lcd display module.



Figure 2.3 LCD Display (24x4)

2.6.4 Arduino Serial Thermal Printer

Mini Thermal Receipt Printer Mini Embedded Receipt Thermal Printer is an embedded thermal printer, which is widely used in various occasions, such as supermarkets, convenience stores, post offices, banks and so on. This is an embedded thermal printer, which is widely used on various occasions, such as supermarkets, convenience stores, post offices, banks, and so on. It is not a normal consumer product, but similar to consumer products. The compact size makes it easy to be embedded in any kind of instrument and meter. With a nice appearance, you can even put it on the table. On the hardware side, we specially choose the TTL interface so that it can be directly controlled by development boards such as Arduino or Raspberry Pi. Figure 2.4 represents the of Serial Thermal Printer.



Figure 2.4 Serial Thermal Printer

2.6.5 Jumper Wire

These female-female Male-female Male-Male jumper wire are handy for making harness or between headers on PCB's. They're 3" (75mm) long and come in a 'strip' of 20 (2pieces of each of ten rainbow colors). The best part is they come in a 20-pin ribbon cable. You can always pull the ribbon wires off to make individual jumpers, or keep them together to make neatly organized wire harnesses. Figure 2.5 represents the jumper wire.

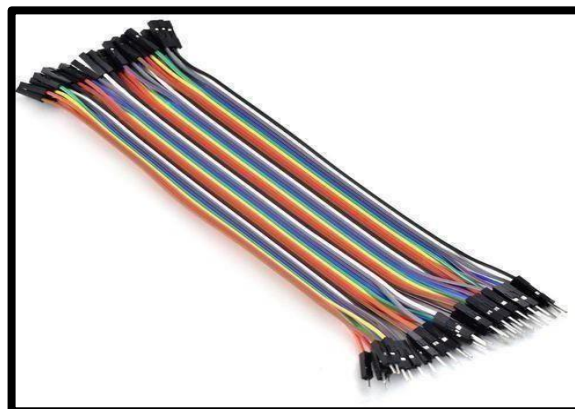


Figure 2.5 Jumper Wire

2.6.6 Bread Board

A breadboard is a circuit board that is used to make temporary circuits. It is a device having electronics and test circuit designs. The electronic elements inside the electronic circuits can be interchanged by inserting the terminals and leads into holes and later connecting it with the help of appropriate wires. The device has stripes of metal below the board that connects the holes placed on the top of the board. Figure 2.6 represents breadboard.

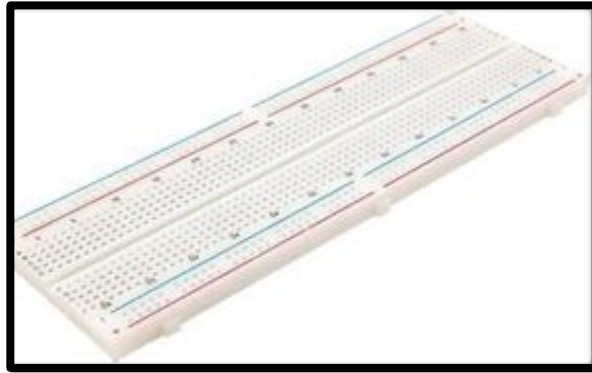


Figure 2.6 Bread Board

2.6.7 Buzzer

A buzzer or beeper is an audio signaling device, which may be mechanical, electromechanical, or piezoelectric. Typical uses of buzzers and beepers include alarm devices, timers, and confirmation of user input such as a mouse click or keystroke. Figure 2.7 represents the buzzer module.



Figure 2.7 Buzzer

2.6.8 Push Button

Push buttons are simple mechanical switches that are used to provide user input to microcontrollers like Arduino. They can be used for various purposes, such as toggling LEDs, starting/stopping processes, or triggering events. Figure 2.8 represents Push button.



Figure 2.8 Push Button

2.6.9 Battery Management System (BMS)

A Battery Management System (BMS) is crucial for protecting lithium-ion and lithium-polymer battery packs. It ensures safe charging and discharging by preventing overcharge, over-discharge, and short circuits. A BMS also balances the charge across cells in a multi-cell battery pack. Figure 2.9 represents Batter Management System Module.

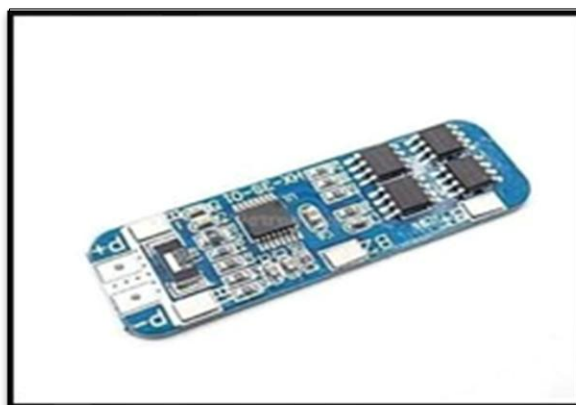


Figure 2.9 Battery Management System

2.6.10 Wi-Fi ESP32 Microcontroller Module

ESP WROOM 32 is a powerful, generic Wi-Fi-BT-BLE MCU module that targets a wide variety of applications, ranging from low-power sensor networks to the most demanding tasks, such as voice encoding, music streaming, and MP3 decoding. At the core of this module is the ESP32 chip, which is designed to be scalable and adaptive. There are 2 CPU cores that can be individually controlled or powered, and the clock frequency is adjustable from 80 MHz to 240MHz. Figure 2.10 represents Wi-Fi ESP32 microcontroller Module



Figure 2.10 ESP32 Wi-Fi Microcontroller

2.6.11 18650 Lithium Ions Cell

An 18650 Lithium Ion Cells is high-capacity cells have been mainly used for flashlights and robotic type applications but with its capability to be used as a drop-in rechargeable cell at 3.37V with a capacity of 600mAh. This is a great battery option for those of you who need a simple to install and replace cell with a lot of juice. These 18650 Cells have a standard discharge current of 0.2C to a maximum of 1C and can handle about 300 charges cycle. Figure 2.11 represents 18650 Lithium Ion Cell.



Figure 2.11 18650 Lithium Ion Cell

2.6.12 12V Power Adapter

12V Power Adapter, AC to DC, 2.1mm X 5.5mm Plug, Regulated 12v 1A, 2A, 2.5A Power Supply Wall Plug for LED Copper String Light, LED Strip Light. This is the most suitable power supply adapter for this project and provide the 1 ~ 1.5A Current that can easily supply power to all components. Figure 2.12 represents 12v Power Adapter.



Figure 2.12 12v Power Adapter

CHAPTER-3 DESIGN & METHODOLOGY

3.1 System Design Overview

The "Digital Electoral System" integrates various hardware and software components to create a seamless and efficient voting process. This section outlines the system's components and their interactions, providing a detailed understanding of its architecture.

Components:

1. **Fingerprint Scanner (R305):** Captures the voter's fingerprint for biometric verification.
2. **Arduino Uno Microcontroller:** Processes the input from the fingerprint scanner and communicates with other components.
3. **LCD Display (24x4):** Provides real-time feedback to the voter, showing the status of their vote.
4. **Thermal Printer:** Prints a physical receipt of the vote, ensuring transparency and verifiability.
5. **Wi-Fi Module (ESP32):** Enables real-time transmission of voting data to a central server.
6. **Mobile Application:** Displays election results to users and allows administrators to manage the system.
7. **Firebase:** Serves as the backend for data storage and real-time synchronization.

Interaction and Data Flow:

1. The voter places their finger on the Fingerprint Scanner, which captures the fingerprint image and sends it to the Arduino Uno for processing.
2. The Arduino Uno verifies the fingerprint against stored templates and, upon successful verification, activates the voting interface on the LCD Display.
3. The voter selects their candidate, and the Arduino Uno records this selection.
4. The Thermal Printer prints a receipt of the vote, which the voter can retain for verification.
5. Simultaneously, the vote data is transmitted via the Wi-Fi Module to the Firebase backend.
6. The Mobile Application retrieves and displays the real-time election results from Firebase, allowing users to monitor the election progress.

3.2 Architectural Design

The architectural design of the "Digital Electoral System" is divided into two main subsections: Hardware Architecture and Software Architecture. This separation provides a clearer understanding of the physical components and the software logic that drive the system.

3.2.1 Hardware Architecture

The hardware architecture comprises several key components that interact to facilitate the voting process:

1. **Fingerprint Scanner (R305):** Captures and processes biometric data for voter authentication.
2. **Arduino Uno Microcontroller:** Serves as the central processing unit, handling inputs from the fingerprint scanner and outputs to the display and printer.
3. **LCD Display (24x4):** Displays real-time information to the voter, such as verification status and voting options.
4. **Thermal Printer:** Provides a printed receipt of the vote for verification and record-keeping.
5. **Wi-Fi Module (ESP32):** Ensures that vote data is transmitted securely to the backend server in real time.

3.2.2 Software Architecture

The software architecture is responsible for the logical operations and data management within the system:

1. **Arduino Firmware:** Controls the interactions between hardware components and manages the logic for vote casting and data transmission.
2. **Mobile Application:** Acts as the user interface for viewing election results and administrative controls.
3. **Firebase Backend:** Handles data storage, real-time updates, and synchronization between the hardware and mobile application.

3.2.3 System Workflow

The system workflow outlines the sequence of operations from voter authentication to the final vote storage. This section provides a clear, structured representation of each step involved in the voting process.

Workflow Steps:

1. **Voter Authentication:**
 - The voter approaches the voting machine and places their finger on the Fingerprint Scanner.
 - The scanner captures the fingerprint and sends it to the Arduino Uno for verification against the database of registered voters.
 - If the fingerprint matches, the system proceeds to the next step; otherwise, the voter is prompted to try again.
2. **Voting Interface Activation:**
 - Upon successful authentication, the LCD Display presents the list of candidates.

- The voter selects their preferred candidate using the interface.
- 3. Vote Recording:**
- The Arduino Uno records the selected vote and updates the internal vote count.
- 4. Vote Receipt Printing:**
- The Thermal Printer prints a receipt of the vote, including the selected candidate and a unique receipt ID for verification.
- 5. Real-time Data Transmission:**
- The recorded vote is transmitted via the Wi-Fi Module to the Firebase Backend for storage and real-time result updates.
- 6. Result Display:**
- The Mobile Application retrieves the latest election results from Firebase and displays them to the users.

3.3 Purposed Methodology

The "Digital Electoral System" was developed using Agile methodology, ensuring flexibility and iterative improvements throughout the project's lifecycle. This section details how Agile principles were applied, including the use of sprints, user stories, and feedback cycles. Figure 3.1 describes the purposed methodology of the "Digital Electoral Process".

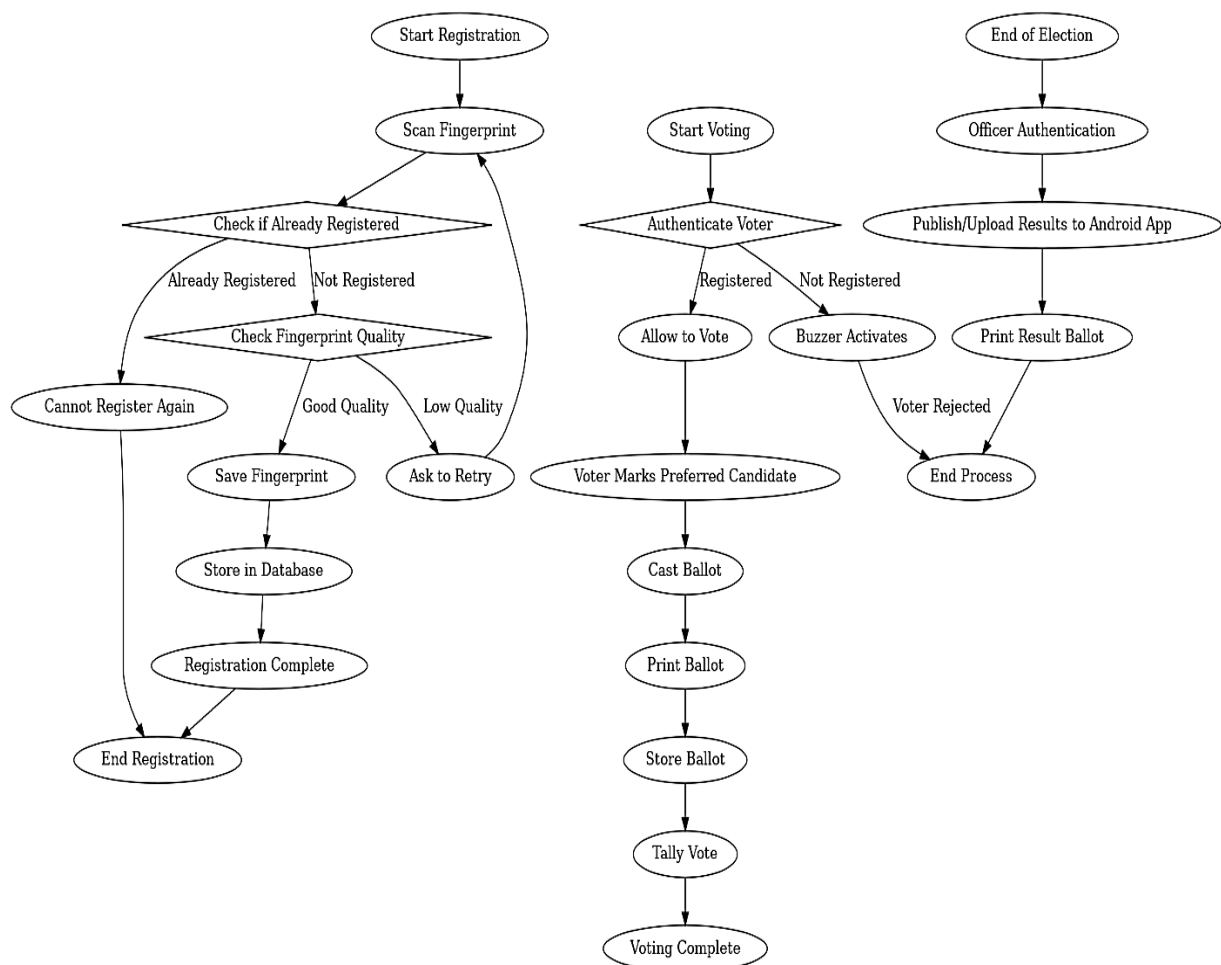


Figure 3.1 Purposed Methodology

3.3.1 Agile Development Process:

1. Sprints:

- The development was divided into two-week sprints, each focusing on specific milestones such as hardware integration, software development, and testing.
- Regular sprint reviews were conducted to assess progress and make necessary adjustments.

2. User Stories:

- User stories were created to capture the requirements from different stakeholders, including voters, administrators, and developers.
- Each story outlined the desired functionality, such as "As a voter, I want to receive a printed receipt of my vote, so I can verify my selection."

3.3.2 Feedback Cycles:

- Continuous feedback was collected from users and stakeholders during each sprint to refine the system's features and usability.
- This iterative process allowed for quick identification and resolution of issues, improving the overall quality of the system.

3.3.3 Development Phases:

The project was divided into several phases, each building upon the previous to ensure a comprehensive and robust system:

1. Phase 1: Requirements Gathering

- Initial meetings with stakeholders to gather requirements and understand the system's needs.

2. Phase 2: System Design

- Designing the system architecture, including hardware and software components.

3. Phase 3: Development

- Implementation of hardware integration, software coding, and backend setup.

4. Phase 4: Testing

- Conducting unit, integration, and system testing to ensure functionality and reliability.

5. Phase 5: Deployment

- Final deployment of the system for real-world use and continuous monitoring.

3.3.4 Visual Aids:

A timeline of the development phases is provided below to illustrate the project's progression:

- **Week 1-2:** Requirements Gathering
- **Week 3-4:** System Design
- **Week 5-8:** Development
- **Week 9-10:** Testing
- **Week 11:** Deployment

3.4 Component Design

3.4.1 Hardware Components

- **Fingerprint Scanner:**
 - Used for voter authentication, ensuring that each voter is uniquely identified.
- **Thermal Printer:**
 - Provides physical receipts for cast votes, enhancing transparency and auditability.
- **Wi-Fi Module:**
 - Facilitates real-time data transfer to the election commission's server.

3.4.2 Software Components

- **Voting Application:**
 - Developed to provide a user-friendly interface for voters and administrators.
- **Database Management:**
 - Securely stores voter information and cast votes.

3.5 Security Measures

The security of the "Digital Electoral System" is paramount, given the sensitive nature of voting data. This section details the specific security measures implemented to protect data integrity and user privacy.

Encryption:

1. AES-256 Encryption:

- All data transmitted between the hardware components, mobile application, and Firebase backend is encrypted using AES-256 (Advanced Encryption Standard with a 256-bit key).
- This ensures that sensitive data, such as vote counts and voter information, is protected from unauthorized access during transmission.

2. SSL/TLS Protocols:

- Secure Sockets Layer (SSL) and Transport Layer Security (TLS) protocols are used to encrypt data transmitted over the network, adding an additional layer of security.

Tamper Detection

1. Checksum Verification:

- Each vote is assigned a unique checksum, which is verified upon receipt by the backend to detect any tampering during transmission.

2. Hardware Security:

- Physical tamper detection mechanisms are integrated into the voting machines to alert administrators in case of unauthorized access or hardware manipulation.

Authentication and Authorization

1. Two-Factor Authentication (2FA):

- Administrators accessing the system's backend are required to use two-factor authentication, enhancing login security.

2. Role-Based Access Control (RBAC):

- Different roles (e.g., voter, administrator) have specific permissions, ensuring that users can only access functionalities relevant to their role.

Data Redundancy and Recovery

1. Data Backups:

- Regular backups of the election data are taken and stored securely to prevent data loss in case of system failure.

2. Redundant Storage:

- Data is stored in multiple geographic locations within Firebase to ensure availability and resilience against data center outages.

3.6 Challenges and Solutions

Several challenges were encountered during the design and implementation phases:

- **Challenge:** Ensuring accurate biometric verification.
 - **Solution:** Calibration of the fingerprint scanner and extensive testing with diverse fingerprints.
- **Challenge:** Real-time data transmission under network constraints.
 - **Solution:** Use of lightweight data packets and optimization of the Wi-Fi module.

3.7 Terminology:

IoT-enabled devices: These are devices that connect to the internet to share and receive data, enhancing the functionality of traditional systems. In this project, components like the Wi-Fi Module and Mobile Application are IoT-enabled, facilitating real-time data synchronization and remote monitoring.

3.8 Working of Components

In our proposed system, we designed an electronic voting system using an Arduino Uno microcontroller, interfacing it with various components to ensure secure and reliable operation. The R305 Fingerprint sensor serves as the primary user authentication device, connected to the Arduino via its digital output pin to pin 7. The sensor receives power directly from the Arduino's 5V power pin. Upon successful fingerprint detection, the Arduino processes the signal and displays the voter's ID and status on a 24.4 LCD module. The LCD's data lines (DB4, DB5, DB6, DB7) are connected to the Arduino's digital pins 5, 4, 3, and 2, respectively, with the Enable pin linked to pin 11 and the RS (Register Select) pin to pin 12.

The LCD backlight is powered by a 5V supply through a 560-ohm current-limiting resistor, and its contrast is adjusted using a 10k potentiometer connected between VEE and the 5V supply. For vote verification, a buzzer connected to pin 8 of the Arduino sounds when the fingerprint matches a registered voter.

Additionally, a thermal printer module, interfaced through software serial communication using the Arduino's PWM pins 9 and 10, prints a confirmation of the registered vote. The thermal printer's TX and RX pins are respectively connected to the Arduino's pin 1 (TX) and a designated RX pin, with their grounds linked.

The ESP32 Module attached and powered by 5V supply through voltage regulator and connected on RX and TX pins are respectively connected to the Arduino's pin 1 (TX) and a designated RX pin, with their grounds linked. It helps to communicate Arduino to Android app via ESP32 and can send and receive information on serial.

This integration of components ensures a seamless and secure voting process, with the Arduino Uno effectively coordinating between the fingerprint sensor, LCD display, buzzer, and thermal printer to provide a functional and user-friendly electronic voting system.

3.9 Working of Project

When the circuit is powered on after uploading code, the LCD displays the registered parties' names and voting button positions. It will display the status of vote count. When a voter places its finger to the sensor it matches the fingerprint of the voter and enable the push button module that allow the person to cast the vote to his desired candidate. When the vote is casted it activate the buzzer one time and send a signal to the thermal printer to print the specific vote. After the completion of voting process, the presiding officer place its thumb and Arduino send a signal to the thermal printer to print list of total vote count. If the fingerprint is not matching its displays invalid voter or if a voter tries to vote again it displays duplicate vote in both case it send signal to buzzer and generate alert three times and not allow the person to vote. When the voting process is complete the presiding officer place its thumb and Arduino send a signal to the ESP32 to send the result to the application portal via Wi-Fi channel using serial communication and ESP32 also parse the information into Json format so it can received successfully at other end.

3.10 Block Diagram

Figure 3.2 describes block diagram of project components. The block diagram (Figure 3.2) illustrates the flow of data within the system: The voter's fingerprint is scanned and sent to the Arduino Uno for verification. Upon successful verification, the LCD Display prompts the voter to cast their vote. The selected vote is recorded by the Arduino Uno and printed by the Thermal Printer. The Wi-Fi Module transmits the vote data to the Firebase Backend. The Mobile Application retrieves the data from Firebase and displays the real-time election results.

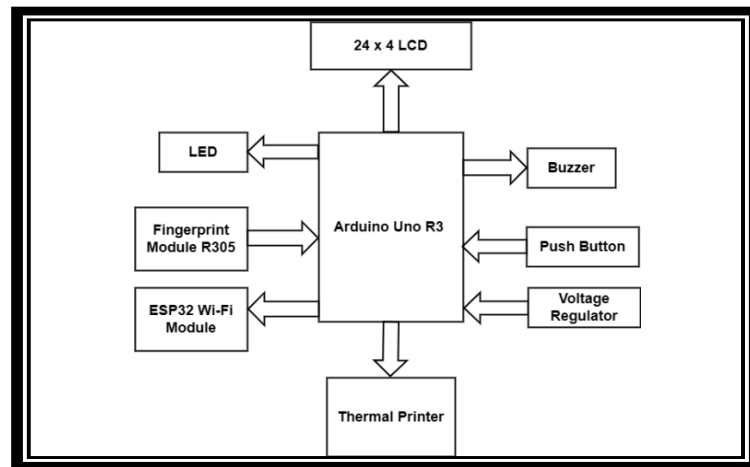


Figure 3.2 Block Diagram

3.11 Use Case Diagram

Figure 3.3 describes use case diagram and role of actors in project. The use case diagram depicts the interactions between users (voters and administrators) and the system. Key use cases include Voter registration and authentication, Casting votes, Result compilation and display, Administrator access for system monitoring.

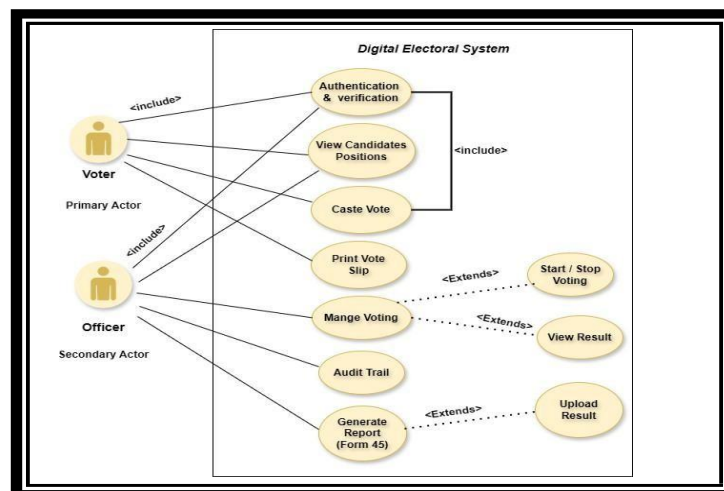


Figure 3.3 Use Case Diagram

3.12 Architecture Diagram

Figure 3.4 describes architecture diagram of project. The architecture diagram for a Digital Electoral System outlines the entire process, starting from user interaction with the interface, moving through image processing and fingerprint enhancement for user identification, followed by feature extraction and matching. It also includes handling any processing failures. Finally, authenticated users' votes are recorded, managed in a database, and results are uploaded. This systematic flow ensures secure and accurate voting.

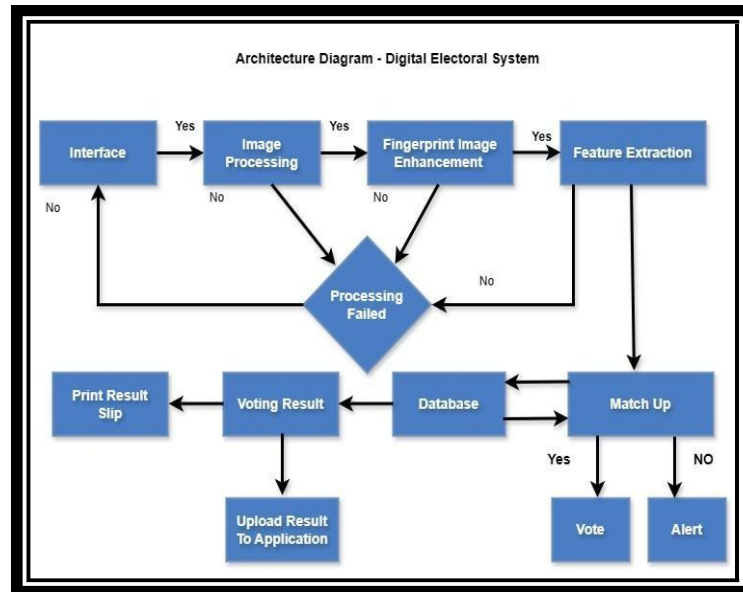


Figure 3.4 Architecture Diagram

3.13 Circuit Diagram

Figure 3.5 describe circuit diagram of project components. The circuit diagram of Digital Electoral System outlines integration of components with Arduino like lcd, buzzer, led, pushbuttons, fingerprint sensor, thermal printer and esp32 wi-fi module. Arduino communicate and control the behavior of components according to provide instruction into the code.

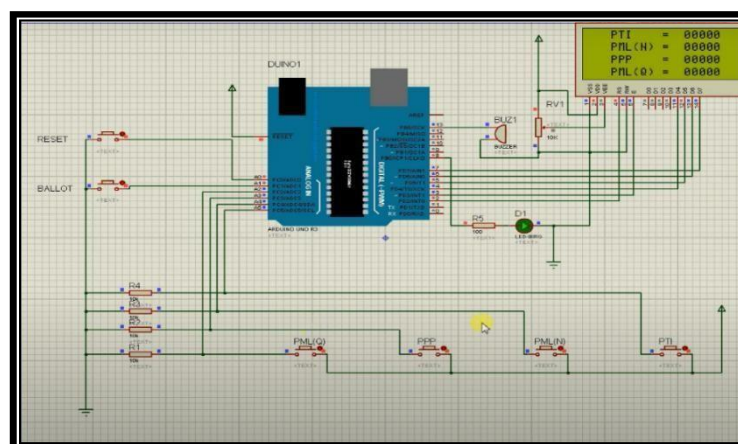


Figure 3.5 Circuit Diagram

3.15 Sequence Diagram

Figure 3.6 describe sequence of project and its working. This sequence diagram illustrates the voting process in the Digital Electoral System. The voter initiates an authentication request via the EVM, and upon approval, selects and confirms a candidate before casting their vote. The admin authenticates and generates the results, which are then uploaded and displayed on the dashboard through the application. The process ensures secure voting, result generation, and transparency for all stakeholders.

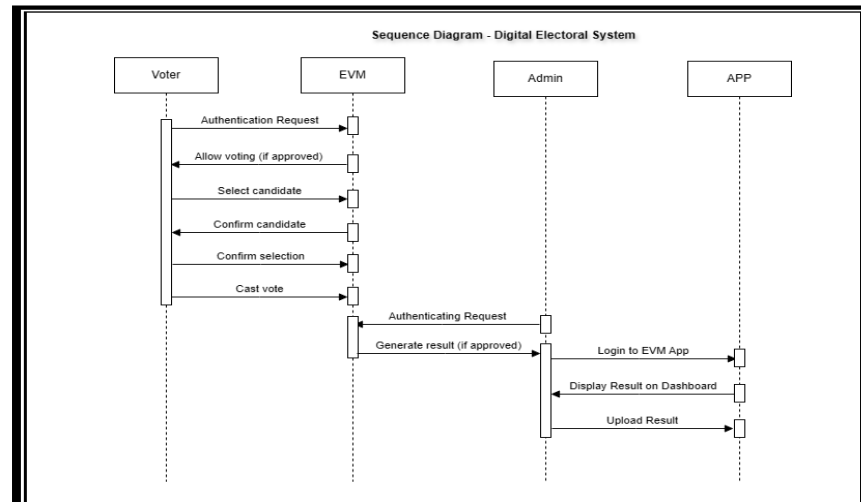


Figure 3.6 Sequence Diagram

3.16 Class Diagram

Figure 3.7 describe class diagram of project. The class diagram for a Digital Electoral System shows how voters authenticate to vote, EVMs (Electronic Voting Machines) record and confirm votes, and each vote links a voter to a candidate. The admin class oversees the entire process, ensuring smooth operations by starting, ending, printing, and uploading data. This organized structure ensures secure and efficient voting.

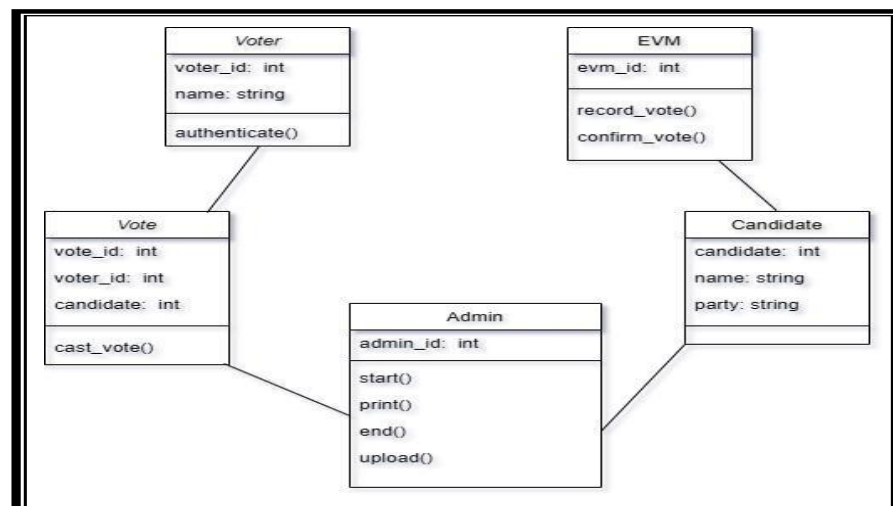


Figure 3.7 Class Diagram

3.17 Flow Chart Fingerprint Enrollment

Figure 3.8 describe Flow chart of fingerprint enrollment process. This flow chart outlines the fingerprint enrollment process of Digital Electoral System. The voter scans their finger if it already exists it does not enroll if it is not enrolled previously, it checks quality of finger print if it good then it saves the fingerprint or else ask user to try again.

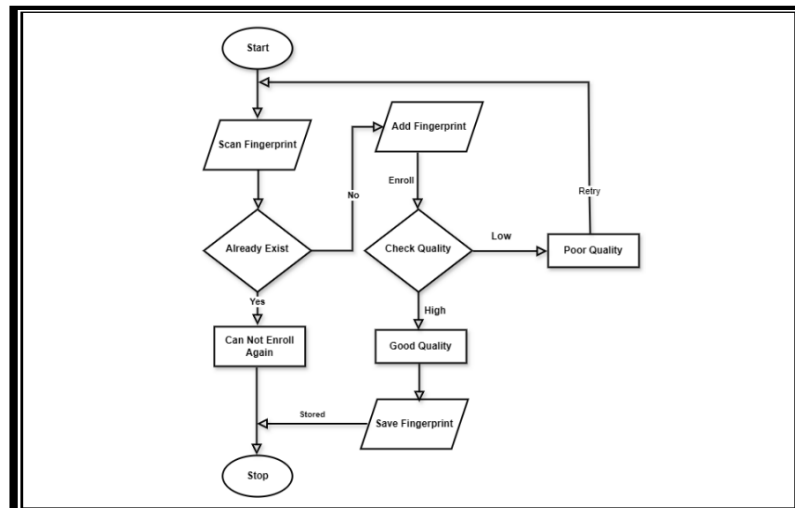


Figure 3.8 Flow Chart Fingerprint Enrollment

3.18 Flow Chart Vote Casting

Figure 3.9 describe Flow chart of fingerprint enrollment process. This flow chart outlines the vote casting process of Digital Electoral System. In this process voter scan their finger if match found then it allows to vote then it select candidate after selecting it confirm the selection and then it cast vote to the candidate. If the voter scans its finger and there is no match found then it does not allow to vote and a buzzer is activated.

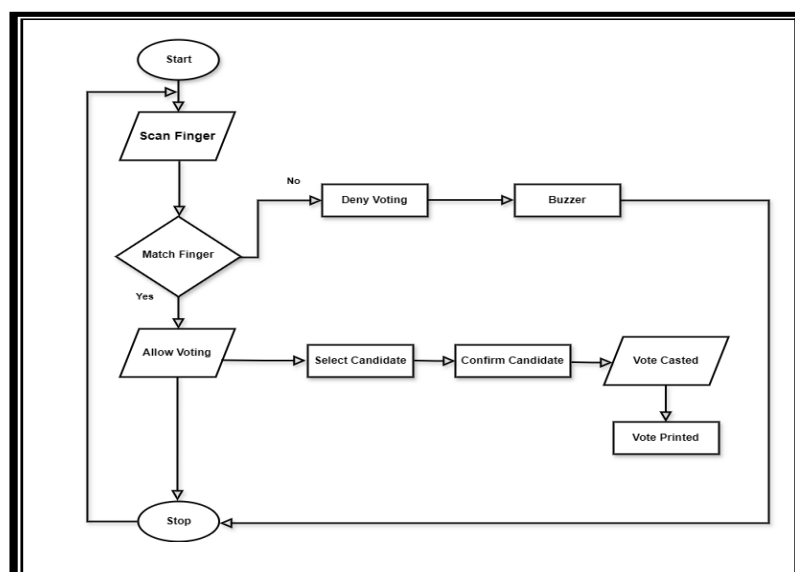


Figure 3.9 Flow Chart Vote Casting

CHAPTER-4 RESULTS AND TESTING

4.1 System Overview

This section provides a detailed overview of the implemented "Digital Electoral System," designed to ensure secure and efficient voting processes. The system integrates biometric authentication to uniquely identify voters, paired with IoT-enabled devices to record, verify, and store votes. This chapter focuses on the outcomes derived from various system tests and functionality evaluations conducted during the project lifecycle.

4.1.1 System Features

- **Biometric Authentication:** Ensures unique voter identification using fingerprints.
- **Real-Time Result Compilation:** The system uploads compiled results to the Election Commission's portal in real time.
- **Tamper-Proof Voting Mechanism:** Includes mechanisms to prevent unauthorized access or manipulation.
- **Voter Turnout Monitoring:** Tracks and reports voter turnout in real time.
- **Cost-Effectiveness:** Reduces the need for manual labor and physical resources, cutting election costs significantly.

4.2 System States

- **EVM Machine States:**

- **Idle State**

Figure 4.1 describe Idle state of Digital Electoral System. This is the idle state of the machine in which all components in an idle state and in ready condition to start the voting process.

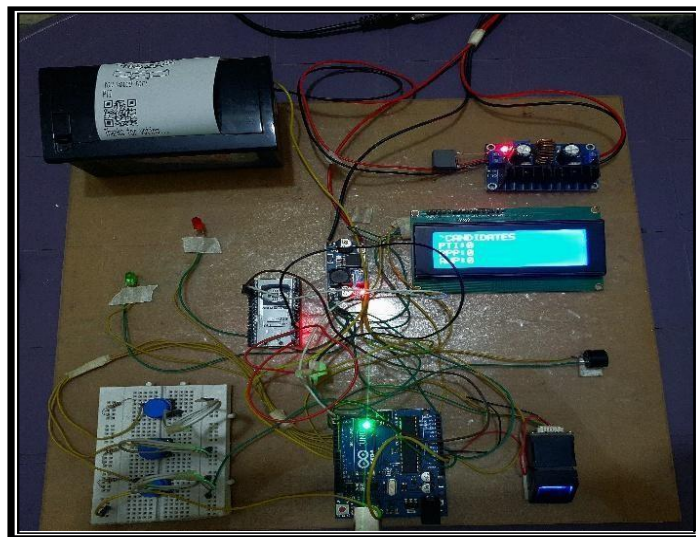


Figure 4.1 Idle State

- **Voter Verification State**

Figure 4.2 describe voter verification state of Digital Electoral System. This is the second state of the machine in which voter scan their finger and machine validate if the person is a registered voter or not and if the voter is registered it display the ID of user otherwise it activates buzzer if a person is not registered and the voting process is start from here.

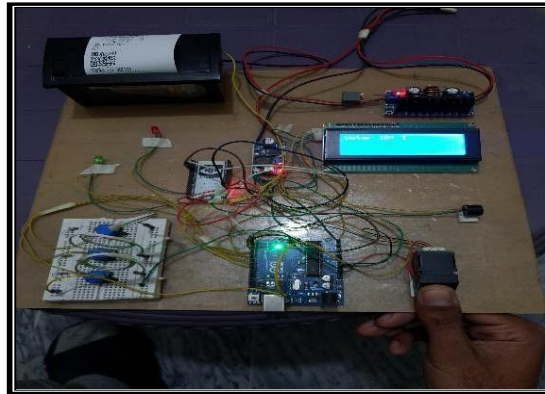


Figure 4.2 Voter Verification State

- **Voting State**

Figure 4.3,4.4,4.5 and 4.6 describe voting state of Digital Electoral System. This is the third state of the machine in which voter select their candidate after scanning their finger then it presses the button to cast the vote then it record the vote of the voter then it print the recoded vote.

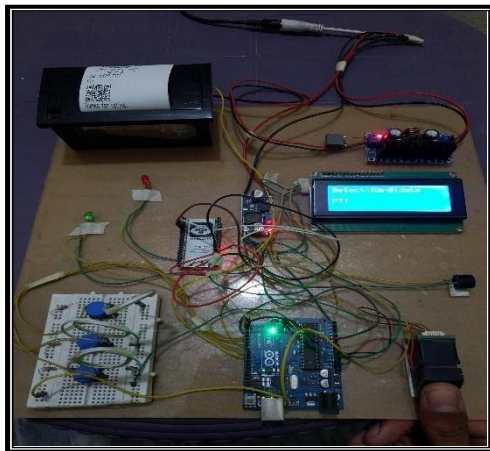


Figure 4.3 Voting State 1

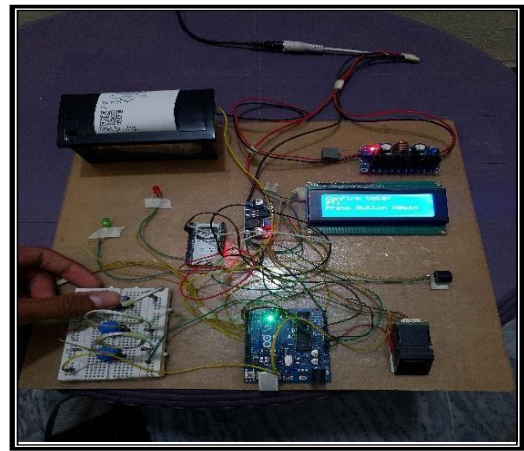


Figure 4.4 Voting State 2

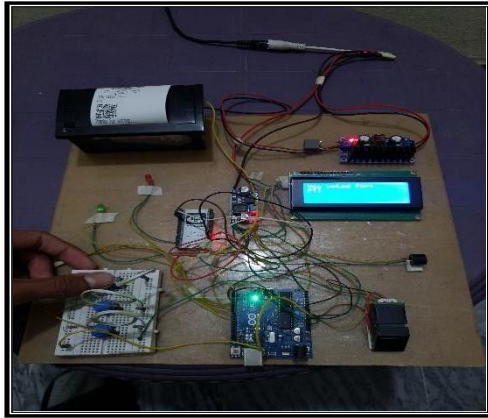


Figure 4.5 Voting State 3

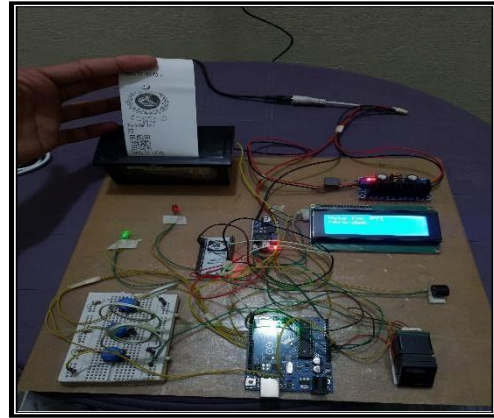


Figure 4.6 Voting State 4

- **Result Compilation State**

Figure 4.7,4.8,4.9 and 4.10 describe Result Compilation state of Digital Electoral System. This is the Fourth state of the machine in which admin scan their fingerprint the machine goes into admin mode then it displays total vote count after that it print the total vote count receipt then it displays the winner.

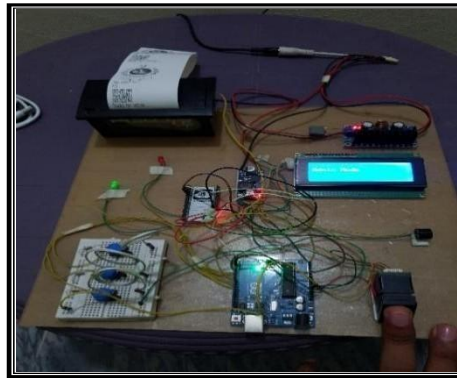


Figure 4.7 Result Compilation State 1

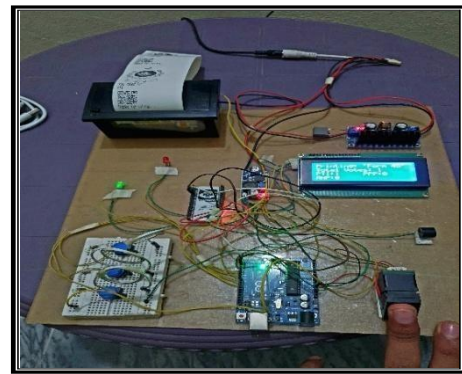


Figure 4.8 Result Compilation State 2

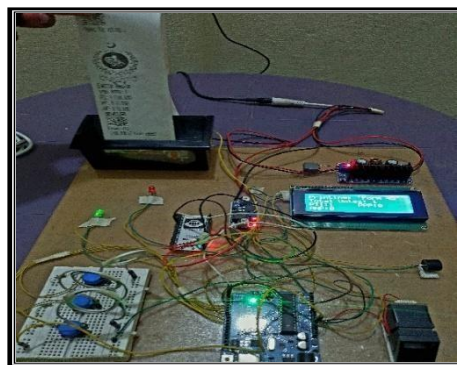


Figure 4.9 Result Compilation State 3

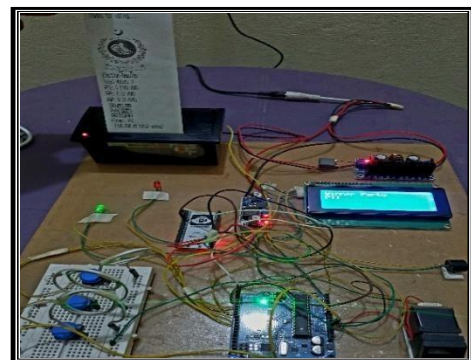


Figure 4.10 Result Compilation State 4

- **Mobile Application States:**

- **Login**

Figure 4.11 describe Login Page of Digital Electoral System application. In login page, user is able to login to our system if his/her details are stored in our database. User has to provide his/her email and password. If the email and password match, he/she transmitted to home page.



Figure 4.11 Login Page

- **Register**

Figure 4.12 describe Register Page of Digital Electoral System application. The user who interacts first time should need to register first. He/She has to enter his Full Name, Cnic Mobile no, Password and Confirm Password and then click on register button. The user transmitted to the login page for login process.

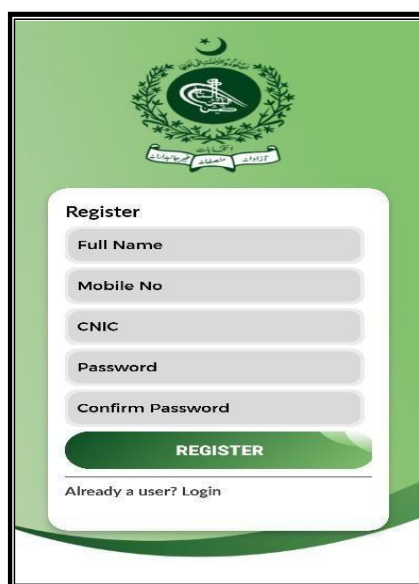


Figure 4.12 Register Page

- **User Dashboard**

Figure 4.13 describe User Dashboard Page of Digital Electoral System application. The user dashboard for the Digital Electoral System displays election results on a green background. The dashboard includes a table listing political parties and their votes. A vote distribution chart shows winner with percentage. Below the chart, there are buttons for uploading images, submitting results, and logging out.

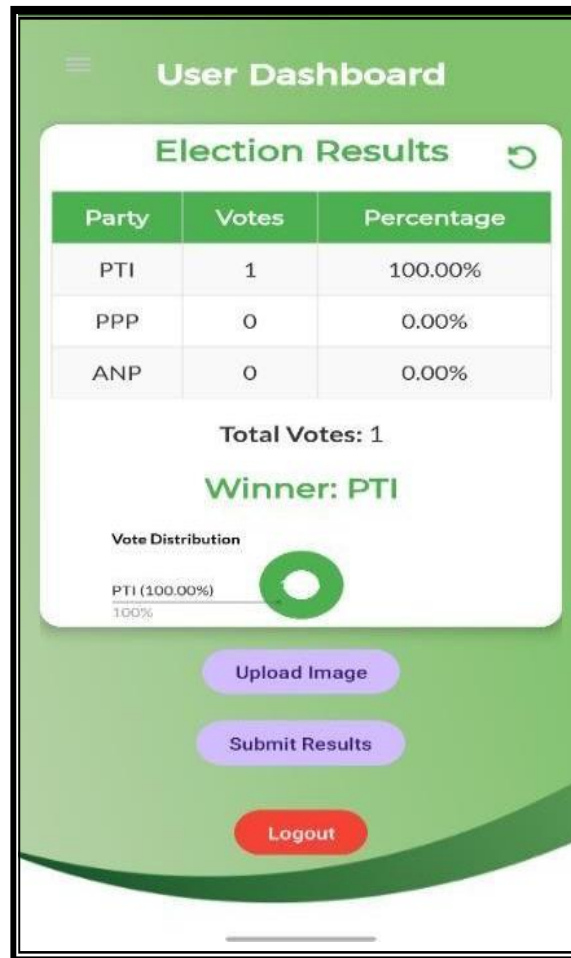


Figure 4.13 User Dashboard

- **Admin Dashboard**

Figure 4.14 describe Admin Dashboard Page of Digital Electoral System application. The Admin Dashboard interface of the Digital Electoral System has two main sections: **User Approvals** and **Election Results**. In the User Approvals section, CNIC numbers are listed for approval, with green "Approve" and red "Reject" buttons below each. The Election Results section displays a table with votes and percentages for different parties: A pie chart visually represents these results. At the bottom, there's a red "Logout" button.

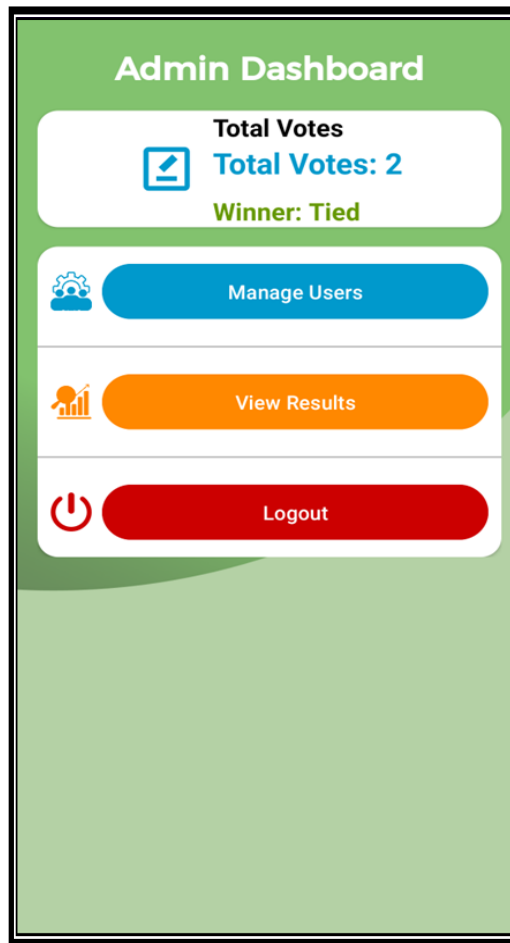


Figure 4.14 Admin Dashboard

- **Data Synchronization with Firebase**

Figure 4.15 describe Data Synchronization with Firebase of Digital Electoral System application. The image shows the Firebase Realtime Database interface. The interface displays a hierarchical JSON structure with three main nodes: **`electionResults`**, **`userLocations`**, and **`users`**. Each node contains specific data entries.

1. electionResults:

- Contains a single entry with the key **`-OGLiAh\_ZUvmrxfbgPnd`**.
- This entry includes:
 - **`ANP\_votes`**: 0
 - **`PPP\_votes`**: 0
 - **`PTI\_votes`**: 1
 - **`TotalVotes`**: 1
 - **`Winner`**: "PTI"
 - **`image`**: A long encoded string representing an image URL.

2. userLocations:

- Contains a single entry with the key **`-3410192304555`**.
- This entry includes:

- `cnic`: "3410192304555"
- `latitude`: 32.1815758
- `longitude`: 74.2133672

3. users:

- Contains a single entry with the key `-OGLiDj4gfmfbJf6vc\_Y`.
- This entry includes:
 - `cnic`: "3410192304555"
 - `fullName`: "Adil Saleem"
 - `mobile`: "03056191383"
 - `password`: "123"

This Figure 4.15 show how data is structured and stored in Firebase Realtime Database, showcasing the hierarchical nature of JSON data and how different types of information (election results, user locations, and user details) can be organized within the database.

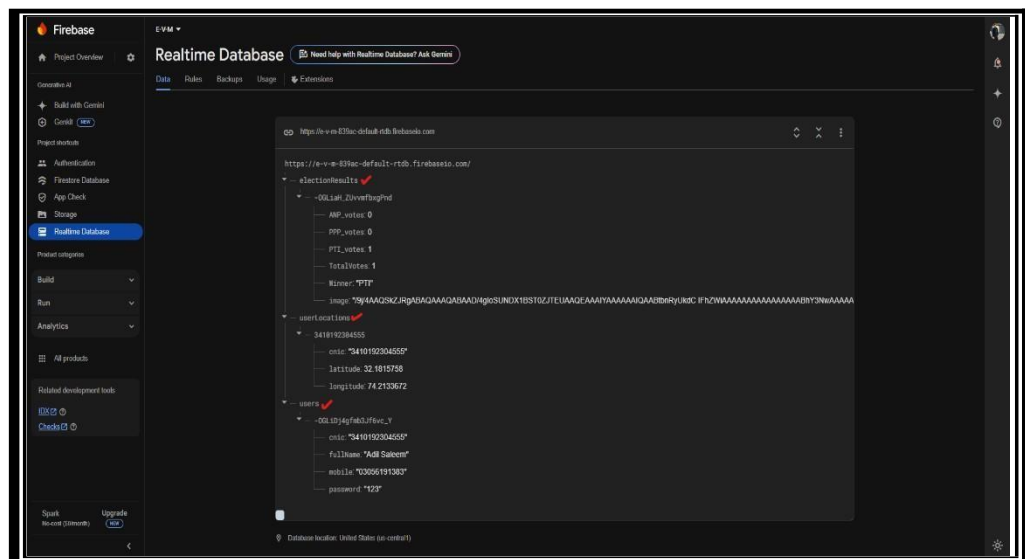


Figure 4.15 Data Synchronization with Firebase

4.3 Testing Methodology

The testing of the "Digital Electoral System" was divided into several phases to evaluate functionality, performance, and security. Each phase involved different test cases to ensure the system's robustness.

4.3.1 Test Environment

The system was tested in a simulated voting environment replicating real-world election scenarios. This included the use of biometric devices, mobile applications, and backend servers for data storage and processing.

4.3.2 Test Phases

1. Unit Testing:

- Ensured individual components, such as the fingerprint module and thermal printer, functioned correctly.
- Example: Verifying that the R305 fingerprint sensor correctly matches stored fingerprints.

2. Integration Testing:

- Evaluated the interaction between hardware components (Arduino Uno, printer, LCD) and the software application.
- Example: Ensuring that voter authentication triggers the correct voting interface on the LCD display.

3. System Testing:

- Tested the complete system in an end-to-end voting process, from voter authentication to result compilation.
- Example: A simulated election with 50 users casting votes.

4. Stress Testing:

- Measured the system's performance under high loads, such as simultaneous voter authentication and voting.
- Example: Simulating 100 users casting votes concurrently.

5. Security Testing:

- Assessed vulnerabilities in voter data storage and transmission.
- Example: Ensuring encrypted storage and secure transmission of biometric data.

4.4 Testing and Results

4.4.1 Login Test Case

Description: To test the functionality of login.

Table 1.7 Login Test Case

Test Case ID	Condition of Test Case	Description	Expected Results	Actual Results
TC1	Email and password are empty.	When user tries to login without entering anything in the id and password fields.	An error message will occur in this case i.e. Both fields must not be empty.	Please fill out this field.
TC2	Email is entered but password is empty.	When user tries to login with only email but did not enter anything in the password field.	An error message will occur in this case i.e. password field must not be empty.	Please fill out this field.

TC3	Email is empty but password is entered	When user tries to login without entering anything in the user's email field but the password field is entered.	An error message will occur in this case i.e. Email field must not be empty.	Please fill out this field.
TC4	Valid email but wrong password	Click on login button with wrong password.	Firestore validate and says Incorrect Password.	Firestore says Incorrect Password.
TC5	Valid email and password	Click on login button with correct email and password.	Bring you to the User dashboard.	User Dashboard appears.

4.4.2 Register Test Case

Description: To test the functionality of Register.

Table 1.8 Register Test Case

Test Case ID	Condition of Test Case	Description	Expected Results	Actual Results
TC1	Full Name, Mobile Number, CNIC, Password, and Confirm Password are empty.	When user tries to register without entering anything in the full name, mobile no, cnic, password and Confirm password fields.	An error message will occur in this case i.e. Every field must not be empty.	Please fill out this field.
TC2	Entered already existing email.	When a user tries to register with a CNIC that is already registered.	An error message will occur in this case i.e. User already exists.	User already exists.
TC3	Enters new cnic, full name, mobile no, and password.	When user tries to register with unique cnic.	Brings you towards the login page.	New users register successfully. Takes you to login page.

4.4.2 User Dashboard Test Case

Description: To test the functionality of the User Dashboard.

Table 1.9 User Dashboard Test Case

Test Case ID	Condition of Test Case	Description	Expected Results	Actual Results
--------------	------------------------	-------------	------------------	----------------

TC1	Navigation Bar	When user tries click on up Navigation Bar icon a side menu appears	Side menu open up Which contain two Buttons.	Sidebar Navigation is working.
TC2	Web View Result loading	When user tries to view result without internet.	An error message will occur Network failure.	Please connect to the network and retry.
TC3	Uploading result using submit button if already submit or session timeout	When a user tries to submit a result after a specific time.	The submit button is disabled and lock for the session	Result already submitted or time limit exceed.

4.5 Test Results

The following table summarizes the outcomes of various tests conducted on the system:

Table 2.0 Test Result

Test Phase	Test Case	Result	Remarks
Unit Testing	Fingerprint Authentication	Pass	Accurate Identification.
Unit Testing	Vote Printing	Pass	Clear and legible vote slips.
Integration Test	LCD and Printer Interaction	Pass	Smooth hardware interaction.
System Testing	Simulated Voting Process	Pass	No errors were detected.
Stress Testing	Concurrent Voting	Pass	Handled 100 users seamlessly.
Security Testing	Biometric Data Encryption	Pass	Data is securely encrypted.

4.6 Firebase Data Management

4.6.1 Data Storage and Retrieval in Firestore

- **Data Storage:** The `storeResultInFirestore` method adds the election results to a Firestore collection named `electionResults`. Each result is stored as a document with key-value pairs representing the votes and the winner.
- **Data Structure:**
 - Collection: `electionResults`
 - Document Fields:
 - ❖ **PTI_votes:** Integer
 - ❖ **PPP_votes:** Integer
 - ❖ **ANP_votes:** Integer
 - ❖ **TotalVotes:** Integer
 - ❖ **Winner:** String
 - ❖ **image:** String (Base64 encoded image)
 - Functionality: This structure allows easy querying and retrieval of election results, supporting operations like fetching all results or specific result entries.

4.6.2 Firebase Realtime Database

- **Data Storage:** The `storeResultInRealtimeDatabase` method saves the results under the path `electionResults` in the Realtime Database, with each result stored under a unique ID generated using `push()`.
- **Data Structure:**
 - **Parent Node:** `electionResults`
 - **Child Nodes:** Each child node represents a result entry with fields similar to Firestore.
 - **Functionality:** The real-time database structure allows for real-time data updates, which is useful for monitoring ongoing elections or other real-time applications.

4.7 Key Observations

- The fingerprint module displayed 98% accuracy in voter authentication.
- Real-time result compilation performed without delays, even under stressful conditions.
- The system's modular design facilitated seamless integration of hardware components.
- Minor delays were observed during simultaneous access by over 100 users, suggesting potential improvements in server scalability.

CHAPTER-5 CONCLUSION AND FUTURE WORK

5.1 Conclusion

The "Digital Electoral System" presents a transformative solution for modernizing electoral processes in Pakistan and similar democracies. This project aimed to address the prevalent challenges in traditional voting systems, such as ballot tampering, voter fraud, delayed results, and high operational costs. By integrating biometric fingerprint recognition with IoT-enabled components like the Arduino Uno, ESP32, thermal printer, and Firebase, the system delivers a secure, efficient, and cost-effective alternative.

The developed system offers:

- **Enhanced Security** through biometric fingerprint authentication, ensuring that each voter is uniquely and accurately identified.
- **Real-time Processing** by instantly compiling and uploading results to a centralized database via a secure cloud service.
- **Transparency and Trust** with printed vote receipts for verifiability and a user-friendly interface for all users, including the less tech-savvy population.
- **Operational Efficiency** by reducing the need for manual labor, paper ballots, and extensive result-counting sessions.
- **Scalability and adaptability** to different election types and sizes, from local council elections to national assemblies.

Throughout various phases of development—including requirement analysis, prototyping, system integration, and testing—the system exhibited robust performance. Test cases confirmed the reliability of the fingerprint module, accuracy of vote printing, secure Firebase synchronization, and resilience under concurrent usage conditions.

5.2 Limitations

Despite its strengths, a few limitations were observed:

- **Limited User Load:** Current testing handled up to 100 concurrent users; scalability for large-scale national elections remains a challenge.
- **Network Dependency:** Real-time features rely on continuous internet access, which may be unavailable in remote or underdeveloped areas.
- **Hardware Constraints:** The use of basic microcontrollers like Arduino Uno restricts the potential for advanced encryption and higher data throughput.

5.3 Future Work

Building upon the current system, several enhancements are proposed:

- **Blockchain Integration:** Implementing blockchain can offer immutable vote logging and tamper-proof audit trails, further increasing trust in the system.
- **Multi-factor Authentication:** Combining biometrics with OTP (One-Time Password) or facial recognition for double-layered voter verification.
- **Offline Mode with Sync:** Allow offline vote collection and automatic sync once connectivity is restored.
- **Web-based Admin Portal:** A centralized admin dashboard for live monitoring, data

analytics, and system health checks.

- **Server Optimization and Load Balancing:** To improve performance under higher voter loads.
- **Accessibility Features:** Include multilingual support, text-to-speech options, and larger text UI for elderly and disabled voters.
- **Compliance & Audit Trail Logs:** Integrate a system for detailed logging and auditability to comply with electoral regulations and enhance public trust.

5.4 Final Remarks

The successful development and implementation of the "Digital Electoral System" is a testament to the viability of using affordable technology to transform national processes. This project bridges the gap between manual voting inefficiencies and modern digital governance. If deployed at scale with support from electoral commissions and government bodies, it can significantly improve the fairness, speed, and transparency of elections in developing nations like Pakistan.

References

- [1] Kumar, D. Ashok, and T. Ummal Sariba Begum. "Electronic voting machine—A review." *International Conference on Pattern Recognition, Informatics and Medical Engineering (PRIME-2012)*. IEEE, 2012.
- [2] Prasad, R. Murali, Polaiah Bojja, and Madhu Nakirekanti. "Aadhar based electronic voting machine using Arduino." *International Journal of Computer Applications* 145.12 (2016): 39-42.
- [3] Srivastava, Shilpa, et al. "Biometric Based Electronic Voting Machine (EVM)." *Authorea Preprints* (2023).
- [4] Arooj, Ansif, and Mohsin Riaz. "Electronic voting with biometric verification offline and hybrid EVMS solution." *2016 Sixth International Conference on Innovative Computing Technology (INTECH)*. IEEE, 2016.
- [5] BalaMurali, A., Potru Sarada Sravanthi, and B. Rupa. "Smart and secure voting machine using biometrics." *2020 Fourth International Conference on Inventive Systems and Control (ICISC)*. IEEE, 2020.
- [6] Rudrappa B. Gujanatti and shivaram N. Tolanur, "A Fingerprint Based Voting System", *International Journal of Engineering Research & Technology (IJERT)*, 2015.
- [7] B. Madan Mohan Reddy and D. SrihariRFID, "Based Biometric Voting Machine Linked To Aadhaar For Safe And Secure Voting", *International Journal of Science Engineering and Technology Research (IJSETR)*, vol. 4, no. 4, April 2015.

APPENDIX A

CODE ARDUINO IDE

MAIN FILE

```
#include <Adafruit_Fingerprint.h>
#include "adalogo.h"
#include "adaqrcode.h"
#include <LiquidCrystal_I2C.h>
#include <Adafruit_Thermal.h>
#include <SoftwareSerial.h>

// Pin definitions
const int button1Pin = 4; // Left navigation
const int button2Pin = 5; // Confirm vote
const int button3Pin = 6; // Right navigation
const int printerRX = 10;
const int printerTX = 11;

int buttonState1 = 0;
int buttonState2 = 0;
int buttonState3 = 0;
int G_led = 8; // choose the pin for the Green LED
int R_led = 9; // choose the pin for the Red Led
const int buzzer = 7;

int id = 0;
int vote_taken = 0;
int totalVotes;
int PTI = 0, PPP = 0, ANP = 0;
String winner_name = "";
String winnerPercentage;

// Array to store voter IDs and a counter for the number of voters
const int MAX_VOTERS = 100; // Maximum number of voters we expect
int voter_ids[MAX_VOTERS];
int voter_count = 0;

// Variables for candidate selection
int currentCandidateIndex = 0;
String candidates[] = {"PTI", "PPP", "ANP"};
int numCandidates = sizeof(candidates) / sizeof(candidates[0]);

// Initialize SoftwareSerial for the printer
SoftwareSerial printerSerial(printerTX, printerRX);
Adafruit_Thermal printer(&printerSerial);
```

```

// Fingerprint sensor pins
SoftwareSerial fingerSerial(2, 3); // RX, TX
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&fingerSerial);

// Initialize LCD
LiquidCrystal_I2C lcd(0x27, 24, 4); // Change the I2C address if needed

void setup() {
  // Initialize Serial communication with ESP32
  Serial.begin(115200); // Assuming ESP32 is connected to the main Serial

  // Initialize software serial for the printer
  printerSerial.begin(9600);
  printer.begin();

  // Initialize the fingerprint sensor
  fingerSerial.begin(57600);
  if (finger.verifyPassword()) {
    Serial.println("Found fingerprint sensor!");
  } else {
    Serial.println("Did not find fingerprint sensor :(");
    while (1) { delay(1); }
  }
  finger.getTemplateCount();
  Serial.print("Sensor contains "); Serial.print(finger.templateCount); Serial.println(" templates");

  // Initialize button pins
  pinMode(buzzer, OUTPUT);
  pinMode(button1Pin, INPUT);
  pinMode(button2Pin, INPUT);
  pinMode(button3Pin, INPUT);

  // Initialize the LCD
  lcd.begin();
  lcd.backlight();
  displayWelcomeMessage();
  delay(2000);
  lcd.clear();
}

```

```

void loop() {
  vote_taken = 0;
  displayScanPrompt();
  displayCandidates();
  enableButtons();

  id = getFingerprintIDez();
  if (id >= 0) {
    displayVoterID(id);

    lcd.setCursor(0, 1);
    if (id == 6) {
      lcd.print("Admin Mode");
      delay(2000);
      lcd.clear();
      determineWinner();
      printResults();
      displayWinner();
      sendResultsToESP32(); // Send results to ESP32
      while (1);
    }
    if (!hasVoterVoted(id)) {
      processVote();
      thankVoter();
    } else {
      handleDuplicateVote();
    }
  }
}

void displayWelcomeMessage() {
  lcd.clear();
  lcd.print("Digital Electoral");
  lcd.setCursor(0, 1);
  lcd.print("System");
  delay(3000);
  lcd.clear();
}

void displayCandidates() {

  lcd.clear();
  lcd.print("CANDIDATES");
  lcd.setCursor(0, 1);
  lcd.print("PTI:");
  lcd.print(PTI);
  lcd.setCursor(0, 2);
  lcd.print("PPP:");
  lcd.print(PPP);
  lcd.setCursor(0, 3);
  lcd.print("ANP:");
  lcd.print(ANP);
}

```

```

void displayScanPrompt() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Please place your");
    lcd.setCursor(0, 1);
    lcd.print("finger");
}
void displayVoterID(int id) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Voter ID: ");
    lcd.print(id);
    delay(2000);
    lcd.clear();
}
void displayVotePrompt() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Give Your vote");
    lcd.setCursor(0, 1);
    lcd.print("Press Button");
    delay(500);
}
void displayCandidateSelection() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Select Candidate");
    lcd.setCursor(0, 2);
    lcd.print(candidates[currentCandidateIndex]);
    delay(300);
}
bool confirmVote() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Confirm Vote:");
    lcd.setCursor(0, 1);
    lcd.print(candidates[currentCandidateIndex]);
    lcd.setCursor(0, 2);
    lcd.print("Press Button Again");

    // Wait for confirmation button press
    long startTime = millis();
    while (millis() - startTime < 5000) { // 5-second window to confirm the vote
        if (digitalRead(button2Pin) == LOW) {
            delay(3000); // Debounce delay
            return true;
        }
    }
    return false; // Time out, return false
}

```

```

void processVote() {
    currentCandidateIndex = 0; // Reset candidate selection
    bool voteRecorded = false;

    while (!voteRecorded) {
        displayCandidateSelection();

        voter_ids[voter_count++] = id; //Detect Duplicate Votes...
        if (digitalRead(button1Pin) == LOW) {
            currentCandidateIndex = (currentCandidateIndex + numCandidates - 1) % numCandidates;
            delay(500); // Debounce delay
        }
        if (digitalRead(button3Pin) == LOW) {
            currentCandidateIndex = (currentCandidateIndex + 1) % numCandidates;
            delay(500); // Debounce delay
        }
        if (digitalRead(button2Pin) == LOW) {
            if (confirmVote()) {
                recordVote(candidates[currentCandidateIndex]);
                handleVote(currentCandidateIndex);
                voteRecorded = true;
                delay(2000);
            } else {
                lcd.clear();
                lcd.setCursor(0, 0);
                lcd.print("Vote Canceled");
                delay(2000);
                // Go back to candidate selection
            }
        }
    }
}

void handleVote(int candidateIndex) {
    if (candidateIndex == 0) {
        PTI++;
    } else if (candidateIndex == 1) {
        PPP++;
    } else if (candidateIndex == 2) {
        ANP++;
    }
    // Activate buzzer and LEDs to indicate successful vote
    activateBuzzerAndLED(1);
    // Display confirmation to the voter
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Vote for ");
    lcd.print(candidates[candidateIndex]);
    lcd.setCursor(0, 1);
    lcd.print("recorded.");
    delay(2000);
}
}

```

```

void recordVote(String candidate) {
  // Display candidate name on LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("You voted for:");
  lcd.setCursor(0, 1);
  lcd.print(candidate);
  delay(2000);

  // Print candidate name using thermal printer
  printer.justify('R');
  printer.printBitmap(266, 282, bitmap_screenshot_gauges);
  printer.setSize('M');
  printer.println("You voted for:");
  printer.println(candidate);

  // Print QR code based on the candidate
  printer.justify('R');
  printer.printBitmap(adaqrwidth, adaqrheight, adaqrdata);
  printer.println(F("Thanks for voting..."));

  printer.feed(2);
}

void thankVoter() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Thanks for your");
  lcd.setCursor(0, 1);
  lcd.print("vote");
  delay(2000);
  activateBuzzerAndLED(1);
}

void sendResultsToESP32() {
  // Format the data as a JSON string
  String data = "{\"totalVotes\": " + String(totalVotes) +
    "\", \"PTI\": " + String(PTI) +
    "\", \"PPP\": " + String(PPP) +
    "\", \"ANP\": " + String(ANP) +
    "\", \"winner\": \"" + winner_name + "\"} \n";

  // Print JSON data to Serial Monitor for verification
  Serial.print("Sending JSON data: ");
  Serial.println(data);

  // Send the JSON string to the ESP32
  Serial.println(data);
}

```

```

void printResults() {
    // Calculate the total number of votes cast
    totalVotes = PTI + PPP + ANP;

    // Calculate the percentage of votes for the winner
    float winnerPercentage = 0.0;
    if (totalVotes > 0) {
        if (winner_name == "PTI") {
            winnerPercentage = (PTI / (float)totalVotes) * 100;
        } else if (winner_name == "PPP") {
            winnerPercentage = (PPP / (float)totalVotes) * 100;
        } else if (winner_name == "ANP") {
            winnerPercentage = (ANP / (float)totalVotes) * 100;
        }
    }
    lcd.clear();
    Serial.println("Election Results");
    // First Row: Display "Printing: 'Form 45'"
    lcd.setCursor(0, 0);
    lcd.print("Printing: 'Form 45'");

    // Second Row: Display Total Votes
    lcd.setCursor(0, 1);
    lcd.print("Total Votes: ");
    lcd.print(totalVotes);
    // Third Row: Display PTI on the left and PPP on the right
    lcd.setCursor(0, 2);
    lcd.print("PTI:");
    Serial.println("PTI:");
    lcd.print(PTI);
    Serial.println(PTI);

    lcd.setCursor(10, 2); // Adjust this value if your screen width differs
    lcd.print("PPP:");
    Serial.println("PPP:");
    lcd.print(PPP);
    Serial.println(PPP);

    // Fourth Row: Display ANP
    lcd.setCursor(0, 3);
    lcd.print("ANP:");
    Serial.println("ANP:");
    lcd.print(ANP);
    Serial.println(ANP);
    Serial.println("TotalVotes:");
    Serial.println(totalVotes);
    Serial.println("Winner: " + winner_name + " \n (" + (winnerPercentage) + "% of total votes)");
    printer.boldOn();
    printer.printBitmap(266, 282, bitmap_screenshot_gauges);
}

```



```

printer.justify('R');
printer.println(F("Election Results"));
printer.boldOff();
printer.justify('R');
printer.println("Total Votes: " + String(totalVotes));
printer.justify('R');
printer.println("PTI: " + String(PTI) + " (" + String((PTI / (float)totalVotes) * 100) + "%)");
printer.justify('R');
printer.println("PPP: " + String(PPP) + " (" + String((PPP / (float)totalVotes) * 100) + "%)");
printer.justify('R');
printer.println("ANP: " + String(ANP) + " (" + String((ANP / (float)totalVotes) * 100) + "%)");
printer.justify('R');
printer.printBitmap(adaqrwidth, adaqrheight, adaqrdata);
printer.justify('R');
printer.setSize('M');
printer.println("Winner: " + winner_name + " \n (" + (winnerPercentage) + "% of total votes)");

printer.feed(3);
activateBuzzerAndLED(1);
delay(3000);
}

void determineWinner() {
  if (PTI > PPP && PTI > ANP) {
    winner_name = "PTI";
  } else if (PPP > PTI && PPP > ANP) {
    winner_name = "PPP";
  } else if (ANP > PTI && ANP > PPP) {
    winner_name = "ANP";
  } else {
    winner_name = "Tied";
  }
}

void displayWinner() {
  lcd.clear();
  lcd.setCursor(0, 0);
  printer.setSize('M');
  lcd.print("Winner Party");
  //printer.println(F("Winner Party"));
  lcd.setCursor(0, 1);
  lcd.print(winner_name);
  //printer.println(winner_name);
  //printer.feed(2);
  delay(500);
}

```

```

void enableButtons() {
    pinMode(button1Pin, INPUT);
    pinMode(button2Pin, INPUT);
    pinMode(button3Pin, INPUT);
}

void disableButtons() {
    pinMode(button1Pin, OUTPUT);
    pinMode(button2Pin, OUTPUT);
    pinMode(button3Pin, OUTPUT);
}

void handleDuplicateVote() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Duplicate Vote");
    lcd.setCursor(0, 1);
    lcd.print("Buttons disabled");
    delay(2000);

    // Disable voting buttons
    digitalWrite(button1Pin, LOW);
    digitalWrite(button2Pin, LOW);
    digitalWrite(button3Pin, LOW);

    activateBuzzerAndLED(3);
}

void activateBuzzerAndLED(int times) {
    for (int i = 0; i < times; i++) {
        digitalWrite(buzzer, HIGH);
        digitalWrite(R_led, HIGH);
        digitalWrite(G_led, LOW);
        delay(1000);
        digitalWrite(buzzer, LOW);
        digitalWrite(R_led, LOW);
        digitalWrite(G_led, HIGH);
        delay(1000);
    }
}

```

```

int getFingerprintIDez() {
    uint8_t p = finger.getImage();
    if (p != FINGERPRINT_OK) return -1;

    p = finger.image2Tz();
    if (p != FINGERPRINT_OK) return -1;

    p = finger.fingerFastSearch();
    if (p != FINGERPRINT_OK) return -1;

    Serial.print("Found ID #"); Serial.print(finger.fingerID);
    Serial.print(" with confidence of "); Serial.println(finger.confidence);
    return finger.fingerID;
}

bool hasVoterVoted(int voter_id) {
    for (int i = 0; i < voter_count; i++) {
        if (voter_ids[i] == voter_id) {
            return true;
        }
    }
    return false;
}

```

HEADER FILE 1

```

#ifndef _adalogo_h_
#define _adalogo_h_

#define adalogo_width 75
#define adalogo_height 75

static const uint8_t PROGMEM bitmap_screenshot_gauges[] = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x06, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x00, 0x01, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x38, 0x00, 0x00, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x00,
};
#endif // _adalogo_h_

```

HEADER FILE 2

```
#ifndef _adaqrcode_h_
#define _adaqrcode_h_

#define adaqrcode_width 135
#define adaqrcode_height 135

static const uint8_t PROGMEM adaqrcode_data[] = {
    0xff, 0xff, 0x00, 0x00, 0x0f, 0xff, 0xff, 0x07, 0xfe, 0x00, 0x03, 0xe0,
    0x07, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00};

#endif // _adaqrcode_h_
```

ESP32 CODE

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// Wi-Fi credentials
const char* ssid = "Adil's PC";
const char* password = "Port:5001";

// Server URL for sending data
const String serverUrl = "http://192.168.1.5/upload.php"; // Android device IP and port

// Buffer size for incoming serial data
const int bufferSize = 256;
```

```

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  Serial.println("Connecting to Wi-Fi...");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connected to Wi-Fi");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  if (Serial.available()) {
    char jsonBuffer[bufferSize];
    int bytesRead = Serial.readBytesUntil('\n', jsonBuffer, bufferSize - 1);
    jsonBuffer[bytesRead] = '\0';

    if (strlen(jsonBuffer) == 0 || jsonBuffer[0] != '{') {
      Serial.println("Skipping non-JSON or empty input");
      return;
    }

    StaticJsonDocument<bufferSize> doc;
    DeserializationError error = deserializeJson(doc, jsonBuffer);

    if (!error) {
      int totalVotes = doc["totalVotes"];
      int PTI = doc["PTI"];
      int PPP = doc["PPP"];
      int ANP = doc["ANP"];
      String winner_name = doc["winner"].as<String>();

      sendToAndroidApp(totalVotes, PTI, PPP, ANP, winner_name);
    } else {
      Serial.print("Failed to parse JSON: ");
      Serial.println(error.c_str());
    }
  }

  delay(10000); // Check every 10 seconds
}

```

```

void sendToAndroidApp(int totalVotes, int PTI, int PPP, int ANP, String winner_name) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverUrl);
        http.addHeader("Content-Type", "application/json");

        StaticJsonDocument<256> jsonDoc;
        jsonDoc["totalVotes"] = totalVotes;
        jsonDoc["PTI"] = PTI;
        jsonDoc["PPP"] = PPP;
        jsonDoc["ANP"] = ANP;
        jsonDoc["winner"] = winner_name;

        String jsonString;
        serializeJson(jsonDoc, jsonString);

        Serial.print("Sending data to server: ");
        Serial.println(jsonString);

        int httpResponseCode = http.POST(jsonString);
        if (httpResponseCode > 0) {
            String response = http.getString();
            Serial.print("Server response: ");
            Serial.println(response);
        } else {
            Serial.print("Error code: ");
            Serial.println(httpResponseCode);
            Serial.print("Error message: ");
            Serial.println(http.errorToString(httpResponseCode));
        }

        http.end();
    } else {
        Serial.println("Error: Not connected to Wi-Fi");
    }
}

```

RESULT CODE

```

{
  "totalVotes": ,
  "PTI": 0,
  "PPP": 0,
  "ANP": 0,
  "winner": ""
}

```

UPLOAD RESULT CODE

```
<?php
// Path to your results.json file
$jsonFilePath = 'results.json';

// Check if the request method is POST
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    // Get the JSON data from the POST request
    $jsonData = file_get_contents('php://input');

    // Decode the incoming JSON data to validate it
    $data = json_decode($jsonData, true);

    // Check if the JSON data is valid
    if ($data === null) {
        echo json_encode(['message' => 'Invalid JSON received.']);
        http_response_code(400); // Bad request
        exit;
    }

    // Option to either replace the file or append the new data
    $replace = true; // Set to false if you want to append

    if ($replace) {
        // Replace the existing results.json file with new data
        file_put_contents($jsonFilePath, json_encode($data, JSON_PRETTY_PRINT));
    } else {
        // Append the new data (merging the new data into the existing file)
        $existingData = json_decode(file_get_contents($jsonFilePath), true);
        $updatedData = array_merge($existingData, $data);
        file_put_contents($jsonFilePath, json_encode($updatedData, JSON_PRETTY_PRINT));
    }

    // Respond to the ESP32 or client that the operation was successful
    echo json_encode(['message' => 'Data successfully updated.']);
} else {
    // If the request is not POST, return a 405 Method Not Allowed
    http_response_code(405);
    echo json_encode(['message' => 'Method not allowed.']);
}
?>
```

UPLOAD RESULT ADMIN CODE

```
<?php
// Database configuration
$host = 'localhost'; // Your database host
$db_name = 'election'; // Your database name
$username = 'your_database_username'; // Your database username
$password = 'your_database_password'; // Your database password

// Create connection
$conn = new mysqli($host, $username, $password, $db_name);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Check if the request method is POST
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    // Get the user ID and other result data
    $user_id = $_POST['user_id'];
    $pti_votes = $_POST['PTI_votes'];
    $ppp_votes = $_POST['PPP_votes'];
    $anp_votes = $_POST['ANP_votes'];
    $total_votes = $_POST['TotalVotes'];
    $winner = $_POST['Winner'];

    // Initialize response array
    $response = array();

    // Process the uploaded image
    if (isset($_POST['image'])) {
        // Decode the base64 string
        $image_data = $_POST['image'];
        $image_data = str_replace('data:image/png;base64,', '', $image_data);
        $image_data = str_replace(' ', '+', $image_data);
        $decoded_image = base64_decode($image_data);

        // Specify the upload directory
        $upload_dir = 'uploads/'; // Make sure this directory exists and is writable
        $image_name = uniqid() . '.png'; // Generate a unique name for the image
        $image_path = $upload_dir . $image_name;
```



```

// Save the image to the server
if (file_put_contents($image_path, $decoded_image)) {
    // Prepare SQL statement to insert the result data
    $stmt = $conn->prepare("INSERT INTO election_results (user_id, PTI_votes, PPP_votes,
ANP_votes, TotalVotes, Winner, image_path) VALUES (?, ?, ?, ?, ?, ?, ?)");
    $stmt->bind_param("iiiiiss", $user_id, $pti_votes, $ppp_votes, $anp_votes, $total_votes,
$winner, $image_path);

    // Execute the query
    if ($stmt->execute()) {
        $response['success'] = true;
        $response['message'] = "Result and image uploaded successfully!";
    } else {
        $response['success'] = false;
        $response['message'] = "Failed to store results: " . $stmt->error;
    }

    // Close the statement
    $stmt->close();
} else {
    $response['success'] = false;
    $response['message'] = "Failed to save image.";
}
} else {
    $response['success'] = false;
    $response['message'] = "No image uploaded.";
}
} else {
    $response['success'] = false;
    $response['message'] = "Invalid request method.";
}
}

// Close the database connection
$conn->close();

// Send the response back to the client
header('Content-Type: application/json');
echo json_encode($response);
?>

```

Application Login Activity

XML Code:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/login_background"
    tools:context=".Login">

    <!-- Logo -->
    <ImageView
        android:id="@+id/logo"
        android:layout_width="201dp"
        android:layout_height="206dp"
        android:src="@drawable/img_logo"
        android:layout_marginTop="20dp"
        android:layout_centerHorizontal="true"
        android:contentDescription="App Logo" />

    <!-- Login Card -->
    <androidx.cardview.widget.CardView
        android:id="@+id/login_card"
        android:layout_width="332dp"
        android:layout_height="355dp"
        android:layout_below="@id/logo"
        android:layout_marginTop="5dp"
        android:layout_centerHorizontal="true"
        android:layout_marginStart="32dp"
        android:layout_marginEnd="32dp"
        app:cardCornerRadius="20dp"
        app:cardElevation="5dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical"
            android:padding="16dp">

            <!-- Login -->
            <TextView
                android:id="@+id/login"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginBottom="20dp"
                android:fontFamily="@font/lato"
                android:gravity="end"
                android:text="Login"
                android:textColor="@color/black_900"
                android:textSize="20dp"
                android:textStyle="bold" />
```

```

<!-- CNIC Field -->
<EditText
    android:id="@+id/cnic_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:layout_marginBottom="16dp"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:backgroundTint="#D9D9D9"
    android:fontFamily="@font/montserratmedium"
    android:hint="CNIC"
    android:inputType="number"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black"
    android:textSize="16sp" />

<!-- Password Field -->
<EditText
    android:id="@+id/password_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:layout_marginBottom="16dp"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:backgroundTint="#D9D9D9"
    android:fontFamily="@font/montserratmedium"
    android:hint="Password"
    android:inputType="textPassword"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black"
    android:textSize="16sp" />

<!-- Forgot Password Link -->
<TextView
    android:id="@+id/forgot_password"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:fontFamily="@font/montserratmedium"
    android:gravity="end"
    android:text="Forget Password?"
    android:textColor="@color/black_900"
    android:textStyle="bold" />

```

```

<!-- Login Button -->
    <android.widget.Button
        android:id="@+id/login_button"
        android:layout_width="300dp"
        android:layout_height="50dp"
        android:layout_marginTop="-1dp"
        android:layout_marginBottom="18dp"
        android:background="@drawable/button_background"
        android:gravity="center"
        android:text="Login"
        android:textColor="@color/white"
        android:textSize="18sp"
        android:textStyle="bold" />

    <!-- Divider Line -->
    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginTop="-5dp"
        android:layout_marginBottom="6dp"
        android:background="@color/black" />

    <!-- Register Link -->
    <TextView
        android:id="@+id/register_link"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:fontFamily="@font/lato"
        android:gravity="left"
        android:text="Not a user? Register"
        android:textColor="@color/gray_800"
        android:textSize="16dp"
        android:textStyle="bold" />

    </LinearLayout>
</androidx.cardview.widget.CardView>

</RelativeLayout>

```

Java Code:

```
package com.example.evm;

import android.Manifest;
import android.app.AlertDialog;
import android.content.Intent;
import android.content.SharedPreferences;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import android.telephony.SmsManager;
import android.util.Log;
import android.view.WindowManager;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;

import java.util.HashMap;
import java.util.Random;
public class Login extends AppCompatActivity {

    private static final int REQUEST_LOCATION_PERMISSION = 100;

    // Fixed admin credentials
    private static final String ADMIN_CNIC = "1234567890123";
    private static final String ADMIN_PASSWORD = "admin123";

    EditText cnicField, passwordField;
    Button loginButton;
    TextView forgotPassword, registerLink;

    SharedPreferences sharedPreferences;
    FusedLocationProviderClient fusedLocationClient;
    Location currentLocation;
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);

    this.getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,
    WindowManager.LayoutParams.FLAG_FULLSCREEN);

    cnicField = findViewById(R.id.cnic_field);
    passwordField = findViewById(R.id.password_field);
    loginButton = findViewById(R.id.login_button);
    forgotPassword = findViewById(R.id.forgot_password);
    registerLink = findViewById(R.id.register_link);

    fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);
    getCurrentLocation();

    sharedPreferences = getSharedPreferences("loginPrefs", MODE_PRIVATE);
    loginButton.setOnClickListener(v -> {
        String cnic = cnicField.getText().toString().trim();
        String password = passwordField.getText().toString().trim();

        if (cnic.isEmpty() || password.isEmpty()) {
            Toast.makeText(Login.this, "Empty input fields", Toast.LENGTH_SHORT).show();
            return;
        }

        if (cnic.equals(ADMIN_CNIC) && password.equals(ADMIN_PASSWORD)) {
            String otp = generateOtp();
            showOtpDialog(cnic, otp, true); // Admin login
        } else {
            checkUserCredentials(cnic, password);
        }
    });

    registerLink.setOnClickListener(v -> startActivity(new Intent(Login.this, Register.class)));
}

private void getCurrentLocation() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) !=
    PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this, new
    String[]{Manifest.permission.ACCESS_FINE_LOCATION}, REQUEST_LOCATION_PERMISSION);
    } else {
        fusedLocationClient.getLastLocation().addOnSuccessListener(this, location -> currentLocation =
    location);
    }
}
}

```

```

private void checkUserCredentials(String cnic, String password) {
    DatabaseReference usersRef = FirebaseDatabase.getInstance().getReference("users");

    usersRef.orderByChild("cnic").equalTo(cnic).get().addOnCompleteListener(task -> {
        if (task.isSuccessful() && task.getResult() != null && task.getResult().getChildrenCount() > 0) {
            DataSnapshot userSnapshot = task.getResult().getChildren().iterator().next();
            String userMobileNumber = userSnapshot.child("mobile").getValue(String.class);
            String registeredPassword = userSnapshot.child("password").getValue(String.class);
            Boolean isBlocked = userSnapshot.child("isBlocked").getValue(Boolean.class);

            if (isBlocked != null && isBlocked) {
                // User is blocked, prevent login
                Toast.makeText(Login.this, "Your account is blocked by admin!",
                Toast.LENGTH_LONG).show();
                return;
            }

            if (registeredPassword != null && registeredPassword.equals(password)) {
                String otp = generateOtp();
                sendSms(userMobileNumber, otp);
                showOtpDialog(cnic, otp, false); // Normal user login
            } else {
                Toast.makeText(Login.this, "Incorrect credentials", Toast.LENGTH_SHORT).show();
            }
        } else {
            Toast.makeText(Login.this, "User not registered", Toast.LENGTH_SHORT).show();
        }
    });
}

// Generate a simple OTP (for example purposes)
private String generateOtp() {
    Random random = new Random();
    int otp = random.nextInt(999999); // Generate a random 6-digit number
    String generatedOtp = String.format("%06d", otp); // Format it to ensure it's 6 digits

    // Log the generated OTP for testing purposes
    Log.d("OTP", "Generated OTP: " + generatedOtp);

    return generatedOtp; // Return the generated OTP
}

private void sendSms(String phoneNumber, String message) {
    try {
        SmsManager smsManager = SmsManager.getDefault();
        smsManager.sendTextMessage(phoneNumber, null, message, null, null);
        Toast.makeText(this, "OTP sent to " + phoneNumber, Toast.LENGTH_SHORT).show();
    } catch (Exception e) {
        Toast.makeText(this, "SMS failed, please try again later!", Toast.LENGTH_SHORT).show();
        e.printStackTrace();
    }
}
}

```

```

private void showOtpDialog(String cnic, String generatedOtp, boolean isAdmin) {
    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    builder.setTitle("Enter OTP");

    final EditText otpInput = new EditText(this);
    builder.setView(otpInput);

    builder.setPositiveButton("Verify", (dialog, which) -> {
        String enteredOtp = otpInput.getText().toString().trim();
        if (enteredOtp.equals(generatedOtp)) {
            saveUserLocation(cnic);
            Toast.makeText(Login.this, "OTP verified successfully", Toast.LENGTH_SHORT).show();
            if (isAdmin) {
                proceedToAdminActivity();
            } else {
                proceedToUserActivity(cnic);
            }
        } else {
            Toast.makeText(Login.this, "Invalid OTP", Toast.LENGTH_SHORT).show();
        }
    });

    builder.setNegativeButton("Cancel", (dialog, which) -> dialog.cancel());
    builder.show();
}

private void saveUserLocation(String cnic) {
    if (currentLocation != null) {
        DatabaseReference userLocationRef =
        FirebaseDatabase.getInstance().getReference("userLocations");
        HashMap<String, Object> locationData = new HashMap<>();
        locationData.put("cnic", cnic);
        locationData.put("latitude", currentLocation.getLatitude());
        locationData.put("longitude", currentLocation.getLongitude());

        userLocationRef.child(cnic).setValue(locationData).addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                Toast.makeText(Login.this, "Location saved successfully", Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(Login.this, "Failed to save location", Toast.LENGTH_SHORT).show();
            }
        });
    } else {
        Toast.makeText(Login.this, "Current location is not available", Toast.LENGTH_SHORT).show();
    }
}

private void proceedToUserActivity(String cnic) {
    sharedPreferences.edit()
        .putBoolean("isLoggedIn", true)
        .putString("userType", "user")
        .putString("user_cnic", cnic)
        .apply();

    startActivity(new Intent(Login.this, UserActivity.class));
    finish();
}

```



```

private void proceedToAdminActivity() {
    sharedPreferences.edit()
        .putBoolean("isLoggedIn", true)
        .putString("userType", "admin")
        .apply();

    startActivity(new Intent(Login.this, AdminActivity.class));
    finish();
}

@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
@NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == REQUEST_LOCATION_PERMISSION) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED)
        {
            getCurrentLocation();
        } else {
            Toast.makeText(this, "Location permission denied", Toast.LENGTH_SHORT).show();
        }
    }
}
}

```

Application Register Activity

XML Code:

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/register_background"
    tools:context=".Register">

    <!-- Logo -->
    <ImageView
        android:id="@+id/logo"
        android:layout_width="201dp"
        android:layout_height="206dp"
        android:src="@drawable/img_logo"
        android:layout_marginTop="16dp"
        android:layout_centerHorizontal="true"
        android:contentDescription="App Logo" />

```

```

<!-- Register Card -->
<androidx.cardview.widget.CardView
    android:id="@+id/Register_card"
    android:layout_width="332dp"
    android:layout_height="455dp"
    android:layout_below="@id/logo"
    android:layout_marginTop="5dp"
    android:layout_centerHorizontal="true"
    android:layout_marginStart="32dp"
    android:layout_marginEnd="32dp"
    app:cardCornerRadius="20dp"
    app:cardElevation="5dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Register -->
        <TextView
            android:id="@+id/Register"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Register"
            android:textColor="@color/black_900"
            android:fontFamily="@font/lato"
            android:textSize="20sp"
            android:gravity="end"
            android:layout_marginBottom="7dp"
            android:textStyle="bold"/>

        <!-- Full Name Field -->
        <EditText
            android:id="@+id/full_name_field"
            android:layout_width="289dp"
            android:layout_height="49dp"
            android:layout_marginBottom="5dp"
            android:background="@drawable/rounded_edittext_with_shadow"
            android:backgroundTint="#D9D9D9"
            android:fontFamily="@font/montserratmedium"
            android:hint="Full Name"
            android:inputType="textPersonName"
            android:padding="12dp"
            android:textColor="@color/black_900"
            android:textColorHint="@color/black"
            android:textSize="16sp" />

```

```

<!-- Mobile Field -->
<EditText
    android:id="@+id/mobile_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:layout_marginBottom="5dp"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:backgroundTint="#D9D9D9"
    android:fontFamily="@font/montserratmedium"
    android:textSize="16sp"
    android:hint="Mobile No"
    android:inputType="number"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black" />

<!-- CNIC Field -->
<EditText
    android:id="@+id/cnic_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:layout_marginBottom="5dp"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:backgroundTint="#D9D9D9"
    android:fontFamily="@font/montserratmedium"
    android:textSize="16sp"
    android:hint="CNIC"
    android:inputType="number"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black" />

<!-- Password Field -->
<EditText
    android:id="@+id/password_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:hint="Password"
    android:inputType="textPassword"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:fontFamily="@font/montserratmedium"
    android:backgroundTint="#D9D9D9"
    android:textSize="16sp"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black"
    android:layout_marginBottom="5dp"/>

```

```

<!-- Confirm Password Field -->
<EditText
    android:id="@+id/confirm_password_field"
    android:layout_width="289dp"
    android:layout_height="49dp"
    android:hint="Confirm Password"
    android:inputType="textPassword"
    android:background="@drawable/rounded_edittext_with_shadow"
    android:fontFamily="@font/montserratmedium"
    android:backgroundTint="#D9D9D9"
    android:textSize="16sp"
    android:padding="12dp"
    android:textColor="@color/black_900"
    android:textColorHint="@color/black"
    android:layout_marginBottom="5dp"/>

<!-- Register Button -->
<android.widget.Button
    android:id="@+id/register_button"
    android:layout_width="300dp"
    android:layout_height="50dp"
    android:layout_marginTop="1dp"
    android:layout_marginBottom="5dp"
    android:background="@drawable/button_background"
    android:text="Register"
    android:textColor="@color/white"
    android:textStyle="bold"
    android:textSize="18sp"
    android:gravity="center" />

<!-- Divider Line -->
<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/black"
    android:layout_marginTop="5dp"
    android:layout_marginBottom="6dp" />

<!-- Login Link -->
<TextView
    android:id="@+id/login_link"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Already a user? Login"
    android:fontFamily="@font/lato"
    android:textSize="16sp"
    android:textColor="@color/gray_800"
    android:gravity="left"
    android:textStyle="bold"/>

</LinearLayout>
</androidx.cardview.widget.CardView>

</RelativeLayout>

```

Java Code:

```
package com.example.evm;

import android.content.Intent;
import android.os.Bundle;
import android.text.InputFilter;
import android.view.View;
import android.view.WindowManager;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;

public class Register extends AppCompatActivity {

    private DatabaseReference databaseReference;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        // Enable full-screen mode
        this.getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,
            WindowManager.LayoutParams.FLAG_FULLSCREEN);

        // Initialize Firebase Database
        databaseReference = FirebaseDatabase.getInstance().getReference("users");
    // Initialize views
    Button btn = findViewById(R.id.register_button);
    EditText fullNameField = findViewById(R.id.full_name_field); // Updated to Full Name
    EditText mobileField = findViewById(R.id.mobile_field); // Updated to Mobile
    EditText cnicField = findViewById(R.id.cnic_field);
    EditText passwordField = findViewById(R.id.password_field);
    EditText confirmPasswordField = findViewById(R.id.confirm_password_field);

    // Set input filter for mobile field to limit length to 11 digits
    mobileField.setFilters(new InputFilter[]{new InputFilter.LengthFilter(11), (source, start, end, dest,
    dstart, dend) -> {
        for (int i = start; i < end; i++) {
            if (!Character.isDigit(source.charAt(i))) {
                return ""; // Reject non-digit characters
            }
        }
        return null; // Accept input
    }});
    }
}
```

```

package com.example.evm;

import android.content.Intent;
import android.os.Bundle;
import android.text.InputFilter;
import android.view.View;
import android.view.WindowManager;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;

public class Register extends AppCompatActivity {

    private DatabaseReference databaseReference;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        // Enable full-screen mode
        this.getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,
            WindowManager.LayoutParams.FLAG_FULLSCREEN);

        // Initialize Firebase Database
        databaseReference = FirebaseDatabase.getInstance().getReference("users");
        // Initialize views
        Button btn = findViewById(R.id.register_button);
        EditText fullNameField = findViewById(R.id.full_name_field); // Updated to Full Name
        EditText mobileField = findViewById(R.id.mobile_field); // Updated to Mobile
        EditText cnicField = findViewById(R.id.cnic_field);
        EditText passwordField = findViewById(R.id.password_field);
        EditText confirmPasswordField = findViewById(R.id.confirm_password_field);

        // Set input filter for mobile field to limit length to 11 digits
        mobileField.setFilters(new InputFilter[]{new InputFilter.LengthFilter(11), (source, start, end, dest,
            dstart, dend) -> {
            for (int i = start; i < end; i++) {
                if (!Character.isDigit(source.charAt(i))) {
                    return ""; // Reject non-digit characters
                }
            }
            return null; // Accept input
        }});
    }
}

```

```

// Set click listener for register button
btn.setOnClickListener(v -> {
    String fullName = fullNameField.getText().toString().trim(); // Updated variable
    String mobile = mobileField.getText().toString().trim(); // Updated variable
    String cnic = cnicField.getText().toString().trim();
    String password = passwordField.getText().toString().trim();
    String confirmPassword = confirmPasswordField.getText().toString().trim();

    // Validate input
    if (fullName.isEmpty() || mobile.isEmpty() || cnic.isEmpty() || password.isEmpty() ||
confirmPassword.isEmpty()) {
        Toast.makeText(Register.this, "All fields are required", Toast.LENGTH_SHORT).show();
    } else if (cnic.length() != 13) {
        Toast.makeText(Register.this, "CNIC must be exactly 13 digits", Toast.LENGTH_SHORT).show();
    } else if (mobile.length() != 11) {
        Toast.makeText(Register.this, "Mobile number must be exactly 11 digits",
Toast.LENGTH_SHORT).show();
    } else if (!password.equals(confirmPassword)) {
        Toast.makeText(Register.this, "Passwords do not match", Toast.LENGTH_SHORT).show();
    } else {
        // Create a new user and save to Firebase
        String userId = databaseReference.push().getKey(); // Generate a unique ID for the user
        User user = new User(fullName, mobile, cnic, password); // Updated User constructor
        if (userId != null) {
            databaseReference.child(userId).setValue(user)
                .addOnCompleteListener(task -> {
                    if (task.isSuccessful()) {
                        Toast.makeText(Register.this, "User registered successfully",
Toast.LENGTH_SHORT).show();
                        // Navigate to the Login activity
                        startActivity(new Intent(Register.this, Login.class));
                    } else {
                        Toast.makeText(Register.this, "Registration failed", Toast.LENGTH_SHORT).show();
                    }
                });
        }
    }
});

// Adjust window insets to avoid overlaps with system bars
View rootView = findViewById(R.id.Register_card);
if (rootView != null) {
    ViewCompat.setOnApplyWindowInsetsListener(rootView, (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return WindowInsetsCompat.CONSUMED; // Return correct insets
    });
}
}
}
}

```

Application User Activity

XML Code:

```
<androidx.drawerlayout.widget.DrawerLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:id="@+id/drawer_layout"
android:layout_width="match_parent"
android:layout_height="match_parent">

<!-- Main Content Layout -->
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/register_background"
    android:padding="16dp">

    <!-- User Panel Title -->
    <TextView
        android:id="@+id/user_panel_title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/user_dashboard"
        android:textSize="24sp"
        android:fontFamily="@font/montserratmedium"
        android:textColor="@color/white"
        android:textStyle="bold"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="8dp" />

    <!-- Timer TextView -->
    <TextView
        android:id="@+id/timer_text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="10:00"
        android:textSize="18sp"
        android:textColor="@color/white"
        android:fontFamily="@font/montserratmedium"
        android:background="@drawable/timer_background"
        android:gravity="center"
        android:padding="6dp"
        android:layout_below="@id/user_panel_title"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="5dp"
        android:elevation="2dp"
        android:shadowColor="#80000000"
        android:shadowDx="2"
        android:shadowDy="2"
        android:shadowRadius="2" />
```



```

<!-- CardView containing the WebView -->
<androidx.cardview.widget.CardView
    android:id="@+id/web_view_card"
    android:layout_width="match_parent"
    android:layout_height="450dp"
    android:layout_below="@id/user_panel_title"
    android:layout_marginTop="50dp"
    android:layout_centerHorizontal="true"
    android:padding="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp">

    <!-- WebView displaying the Election Results Table -->
    <WebView
        android:id="@+id/results_webview"
        android:layout_width="match_parent"
        android:layout_height="456dp"
        android:background="#F5F5F5" />
</androidx.cardview.widget.CardView>

<!-- Additional elements unchanged -->
<ImageView
    android:id="@+id/image_preview"
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:layout_below="@id/web_view_card"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="16dp"
    android:contentDescription="@string/todo"
    android:visibility="gone" />

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@id/image_preview"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="16dp"
    android:orientation="horizontal"
    android:gravity="center"
    android:padding="8dp">

    <!-- Upload Image Button -->
    <Button
        android:id="@+id/upload_image_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/upload_image"
        android:drawableStart="@drawable/ic_upload"
        android:drawablePadding="8dp"
        android:layout_marginEnd="16dp"
        android:textColor="@color/white"
        android:paddingStart="16dp"
        android:paddingEnd="16dp" />

```

```

<!-- Submit Results Button -->
    <Button
        android:id="@+id/submit_results_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/submit_results"
        android:drawableStart="@drawable/ic_submit"
        android:drawablePadding="8dp"
        android:textColor="@color/white"
        android:paddingStart="16dp"
        android:paddingEnd="16dp" />
</LinearLayout>

<Button
    android:id="@+id/user_logout_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/logout"
    android:textColor="#FFF"
    android:backgroundTint="#F44336"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="650dp" />
</RelativeLayout>
</androidx.drawerlayout.widget.DrawerLayout>

```

Java Code:

```

package com.example.evm;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.content.SharedPreferences;
import android.net.Uri;
import android.os.Bundle;
import android.provider.MediaStore;
import android.util.Log;
import androidx.annotation.Nullable;
import android.os.CountDownTimer;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.RequestQueue;
import com.android.volley.toolbox.Volley;
import com.google.firebase.FirebaseApp;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.storage.FirebaseStorage;
import com.google.firebase.database.DatabaseReference;

```

```

import com.google.firebase.database.FirebaseDatabase;
import java.util.HashMap;
import java.util.Map;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.util.Base64;
import android.view.WindowManager;
import com.android.volley.Request;
import com.android.volley.toolbox.JsonObjectRequest;
import org.json.JSONObject;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;
import java.util.Objects;
public class UserActivity extends AppCompatActivity {

    private WebView webView;
    private Button uploadImageButton;
    private Button submitButton;
    private ImageView uploadedImageView;
    private RequestQueue requestQueue;
    private static final String RESULTS_URL = "http://192.168.1.5/results.json"; // Replace with your
actual URL
    private SharedPreferences sharedPreferences;
    private boolean resultSubmitted = false; // To track if the result has been submitted
    private String timeReceived;

    // Declare a constant for image selection
    private static final int CAMERA_REQUEST = 2;
    private static final int PICK_IMAGE_REQUEST = 1; // Code for picking an image
    private Bitmap selectedImageBitmap;

    // Firebase Firestore and Storage
    private FirebaseFirestore firestore;
    // Timer variables
    private TextView timerText;
    private CountdownTimer countdownTimer;
    private static final int TIME_LIMIT_MILLIS = 120000; // 2 minutes

    @SuppressWarnings("SetJavaScriptEnabled")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        FirebaseApp.initializeApp(this);
        setContentView(R.layout.activity_user);

        this.getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,
            WindowManager.LayoutParams.FLAG_FULLSCREEN); // Enable full screen

```

```

// Initialize components
webView = findViewById(R.id.results_webview);
Button logoutButton = findViewById(R.id.user_logout_button);
uploadImageButton = findViewById(R.id.upload_image_button);
submitButton = findViewById(R.id.submit_results_button);
uploadedImageView = findViewById(R.id.image_preview);
requestQueue = Volley.newRequestQueue(this);

// Initialize Firebase Firestore and Storage
firestore = FirebaseFirestore.getInstance();
FirebaseStorage.getInstance().getReference();

// Setup WebView
WebSettings webSettings = webView.getSettings();
webSettings.setJavaScriptEnabled(true);
webView.setWebViewClient(new WebViewClient());
webView.addJavascriptInterface(new WebAppInterface(), "Android");

// Initialize shared preferences for session management
sharedPreferences = getSharedPreferences("loginPrefs", MODE_PRIVATE);

// Load Election Results from the server
fetchElectionResults();

// Initialize the timer TextView
timerText = findViewById(R.id.timer_text);
// Start the countdown timer
startCountdownTimer();
// Start timer to enforce the time limit
//startSubmissionTimeLimit();

// Logout button functionality
logoutButton.setOnClickListener(v -> {
    sharedPreferences.edit().clear().apply();
    Intent intent = new Intent(UserActivity.this, Wellcome.class);
    startActivity(intent);
    finish();
});
// Upload image button functionality
uploadImageButton.setOnClickListener(v -> {
    // Open gallery or camera dialog
    showImagePickerDialog();
});
// Submit button functionality
submitButton.setOnClickListener(v -> {
    if (resultSubmitted) {
        Toast.makeText(UserActivity.this, "Result already submitted", Toast.LENGTH_SHORT).show();
        return;
    }
    // Fetch the current results from the JSON URL
    fetchCurrentResultsAndUpload();
});
}

```

```

// Start the countdown timer
private void startCountdownTimer() {
    countdownTimer = new CountdownTimer(TIME_LIMIT_MILLIS, 1000) {
        @Override
        public void onTick(long millisUntilFinished) {
            // Update the timer text every second
            long minutes = (millisUntilFinished / 1000) / 60;
            long seconds = (millisUntilFinished / 1000) % 60;
            String timeLeftFormatted = String.format(Locale.getDefault(), "%02d:%02d", minutes, seconds);
            timerText.setText(timeLeftFormatted);
        }

        @Override
        public void onFinish() {
            // Handle timer finish (e.g., disable the submit button)
            timerText.setText("00:00");
            submitButton.setEnabled(false);
            Toast.makeText(UserActivity.this, "Submission time limit exceeded",
                Toast.LENGTH_SHORT).show();
        }
    }.start();
}

@Override
protected void onDestroy() {
    super.onDestroy();
    // Cancel the timer to avoid memory leaks
    if (countdownTimer != null) {
        countdownTimer.cancel();
    }
}

//Image Selection Logic.....
private void showImagePickerDialog() {
    // Here, you could show a dialog for the user to choose between gallery or camera.
    // For simplicity, let's open the gallery.
    Intent intent = new Intent(Intent.ACTION_PICK,
        MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
    startActivityForResult(intent, PICK_IMAGE_REQUEST);
}

```

```

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if (resultCode == RESULT_OK) {
        if (requestCode == PICK_IMAGE_REQUEST && data != null && data.getData() != null) {
            // Add a variable to store the selected image URI
            Uri selectedImageUri = data.getData();

            try {
                selectedImageBitmap = MediaStore.Images.Media.getBitmap(getContentResolver(),
selectedImageUri);
                uploadedImageView.setImageBitmap(selectedImageBitmap);

                // Disable upload image button after selection
                uploadImageButton.setEnabled(false);
            } catch (IOException e) {
                e.printStackTrace(); // Log exception
            }
        } else if (requestCode == CAMERA_REQUEST && data != null) {
            selectedImageBitmap = (Bitmap) Objects.requireNonNull(data.getExtras()).get("data");
            uploadedImageView.setImageBitmap(selectedImageBitmap);

            // Disable upload image button after selection
            uploadImageButton.setEnabled(false);
        }
    }
}

// Method to fetch current results and upload to Firebase
private void fetchCurrentResultsAndUpload() {
    JsonObjectRequest jsonObjectRequest = new JsonObjectRequest(
        Request.Method.GET,
        RESULTS_URL,
        null,
        response -> {
            // Convert the response to a Map
            Map<String, Object> resultData = new HashMap<>();
            resultData.put("PTI_votes", response.optInt("PTI", 0));
            resultData.put("PPP_votes", response.optInt("PPP", 0));
            resultData.put("ANP_votes", response.optInt("ANP", 0));
            resultData.put("Total Votes", response.optInt("total Votes", 0));
            resultData.put("Winner", response.optString("winner", ""));
            if (selectedImageBitmap != null) {
                String imageString = bitmapToBase64(selectedImageBitmap);
                resultData.put("image", imageString);
            }

            // Upload to Firestore and Realtime Database
            storeResultInFirestore(resultData);
            storeResultInRealtimeDatabase(resultData);
        }
    );
}

```

```

// Lock result submission after success
    resultSubmitted = true;
    submitButton.setEnabled(false);
    Toast.makeText(UserActivity.this, "Result submitted successfully!",
Toast.LENGTH_SHORT).show();
    },
    error -> Toast.makeText(UserActivity.this, "Failed to fetch current results",
Toast.LENGTH_SHORT).show()
    );

    requestQueue.add(jsonObjectRequest);
}

private String bitmapToBase64(Bitmap bitmap) {
    ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
    bitmap.compress(Bitmap.CompressFormat.JPEG, 80, byteArrayOutputStream); // Compress image
for efficient storage
    byte[] imageBytes = byteArrayOutputStream.toByteArray();
    return Base64.encodeToString(imageBytes, Base64.DEFAULT);
}

// Store Real-Time Result
private void storeResultInRealtimeDatabase(Map<String, Object> resultData) {
    DatabaseReference databaseReference =
FirebaseDatabase.getInstance().getReference("electionResults");
    String resultId = databaseReference.push().getKey();

    if (resultId != null) {
        databaseReference.child(resultId).setValue(resultData)
            .addOnSuccessListener(aVoid -> {
                Log.d("Firebase", "Result saved in Realtime Database successfully.");
                Toast.makeText(UserActivity.this, "Result saved in Realtime Database",
Toast.LENGTH_SHORT).show();
            })
            .addOnFailureListener(e -> {
                Log.e("Firebase", "Error saving result in Realtime Database: " + e.getMessage());
                Toast.makeText(UserActivity.this, "Error saving result in Realtime Database: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
            });
    } else {
        Log.e("Firebase", "Failed to generate unique ID for Realtime Database.");
        Toast.makeText(UserActivity.this, "Failed to generate unique ID",
Toast.LENGTH_SHORT).show();
    }
}

// Store result in Firestore
private void storeResultInFirestore(Map<String, Object> resultData) {
    firestore.collection("electionResults")
        .add(resultData)
        .addOnSuccessListener(documentReference -> Toast.makeText(UserActivity.this, "Result saved
in Firestore", Toast.LENGTH_SHORT).show())
        .addOnFailureListener(e -> Toast.makeText(UserActivity.this, "Error saving result in Firestore",
Toast.LENGTH_SHORT).show());
}

```

```

private void fetchElectionResults() {
    JsonObjectRequest jsonObjectRequest = new JsonObjectRequest(
        Request.Method.GET,
        RESULTS_URL,
        null,
        response -> {
            timeReceived = getCurrentTime(); // Record the time the result was received
            String htmlData = formatElectionResult(response);
            webView.loadDataWithBaseURL(null, htmlData, "text/html", "UTF-8", null);
        },
        error -> Toast.makeText(UserActivity.this, "Failed to load results",
        Toast.LENGTH_SHORT).show()
    );
    requestQueue.add(jsonObjectRequest);
}

// Get the current time
private String getCurrentTime() {
    SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss", Locale.getDefault());
    return sdf.format(new Date());
}

public String getTimeReceived() {
    return timeReceived;
}

public void setTimeReceived(String timeReceived) {
    this.timeReceived = timeReceived;
}

// Create a JavaScript interface class
public class WebAppInterface {
    @android.webkit.JavascriptInterface
    public void refreshResults() {
        runOnUiThread(UserActivity.this::fetchElectionResults);
    }
}

// Convert the drawable image to a base64 string
public String getBase64FromDrawable(int drawableId) {
    Bitmap bitmap = BitmapFactory.decodeResource(getResources(), drawableId);
    ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
    bitmap.compress(Bitmap.CompressFormat.PNG, 100, outputStream);
    byte[] byteArray = outputStream.toByteArray();
    return Base64.encodeToString(byteArray, Base64.DEFAULT);
}

private String formatElectionResult(JSONObject json) {
    try {
        int totalVotes = json.getInt("totalVotes");
        int ptiVotes = json.getInt("PTI");
        int pppVotes = json.getInt("PPP");
        int anpVotes = json.getInt("ANP");
    }
}

```



```

@SuppressLint("DefaultLocale") String ptiPercentage = String.format("%.2f%%", (ptiVotes / (float)
totalVotes) * 100);
    @SuppressLint("DefaultLocale") String pppPercentage = String.format("%.2f%%", (pppVotes /
(float) totalVotes) * 100);
    @SuppressLint("DefaultLocale") String anpPercentage = String.format("%.2f%%", (anpVotes /
(float) totalVotes) * 100);

    // Get base64 image for refresh button
    String refreshBase64 = getBase64FromDrawable(R.drawable.refresh); // assuming refresh.png is
in drawable

    String html = "<html><head>" +
        "<link
href='https://fonts.googleapis.com/css2?family=Lato:wght@400;700&family=Montserrat:wght@500&di
splay=swap' rel='stylesheet'>" +
        "<style>" +
        ".refresh-btn { position: absolute; top: 0; right: 0; margin: 10px; cursor: pointer; }" +
        "</style>" +
        // Google Circle Chart (Pie Chart)
        "<script type='text/javascript' src='https://www.gstatic.com/charts/loader.js'></script>"
+
        "<script type='text/javascript'>" +
        "google.charts.load('current', {'packages':['corechart']});" +
        "google.charts.setOnLoadCallback(drawChart);" +
        "function drawChart() {" +
        "var data = google.visualization.arrayToDataTable([[" +
        "['Party', 'Percentage']," +
        "['PTI (" + ptiPercentage + ")", " + ptiVotes + "], " +
        "['PPP (" + pppPercentage + ")", " + pppVotes + "], " +
        "['ANP (" + anpPercentage + ")", " + anpVotes + "]]" +
        "]);" +
        "var options = {" +
        "title: 'Vote Distribution', " +
        "pieHole: 0.4, " +
        "pieSliceText: 'percentage', " +
        "legend: { position: 'labeled' }, " +
        "fontName: 'Lato', " +
        "fontSize: 12, " +
        "colors: ['#4CAF50', '#FFC107', '#F44336'], " +
        "width: '50%', height: 100 " + // Set the width and height to match the table
        "};" +
        "var chart = new google.visualization.PieChart(document.getElementById('piechart'));" +
        "chart.draw(data, options);" +
        "}" +
        "</script>" +
        "</head><body style='font-family:Lato, sans-serif; color:#333;'>" +

```

```

// Title and Refresh Button
    "<div style='position:relative;'>" +
    "<h2 style='text-align:center; color:#4CAF50; font-family:Montserrat, sans-serif;'>Election
Results</h2>" +
    "<img src='data:image/png;base64," + refreshBase64 + "' alt='Refresh' class='refresh-btn'
style='width:24px; height:24px;' onclick='Android.refreshResults()' />" +
    "</div>" +
    "<div style='width:100%; margin:auto; max-width:600px;'>" +
    "<table style='width:100%; border-collapse: collapse;'>" +
    "<thead style='background-color:#f2f2f2;'>" +
    "<tr><th style='padding:12px; border:1px solid #ddd; background-color:#4CAF50;
color:#fff; font-family:Montserrat, sans-serif;'>Party</th>" +
    "<th style='padding:12px; border:1px solid #ddd; background-color:#4CAF50; color:#fff;
font-family:Montserrat, sans-serif;'>Votes</th>" +
    "<th style='padding:12px; border:1px solid #ddd; background-color:#4CAF50; color:#fff;
font-family:Montserrat, sans-serif;'>Percentage</th></tr>" +
    "</thead><tbody>" +

    // PTI Row
    "<tr style='background-color:#f9f9f9;'>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'>PTI</td>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + ptiVotes + "</td>"
+
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + ptiPercentage +
"</td>" +
    "</tr>" +

    // PPP Row
    "<tr>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'>PPP</td>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + pppVotes + "</td>"
+
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + pppPercentage +
"</td>" +
    "</tr>" +

    // ANP Row
    "<tr style='background-color:#f9f9f9;'>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'>ANP</td>" +
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + anpVotes + "</td>"
+
    "<td style='padding:12px; text-align:center; border:1px solid #ddd;'> + anpPercentage +
"</td>" +
    "</tr>" +
    "</tbody></table>" +
    "<div style='margin-top:20px; text-align:center;'>" +
    "<p style='font-size:18px; color:#333; font-family:Lato, sans-serif;'><strong>Total Votes:
</strong>" + totalVotes + "</p>" +
    "<p style='font-size:22px; font-weight:bold; color:#4CAF50; font-family:Montserrat, sans-
serif;'>Winner: " + json.getString("winner") + "</p>" +
    "</div>" +

```

```
// Google Circle Chart (Pie Chart) directly under Winner Name with table size
    "<div id='piechart' style='width:100%; max-width:600px; height:400px;
margin:auto;'></div>" +
    "</div></body></html>";

    return html;
} catch (Exception e) {
    e.printStackTrace();
    return "<html><body><p>Error loading results</p></body></html>";
}
}
}
```

Application Admin Activity

XML Code:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@drawable/welcome_bg"
tools:context=".AdminActivity">

    <!-- Admin Dashboard Title -->
    <TextView
        android:id="@+id/admin_dashboard_title"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Admin Dashboard"
        android:textSize="26sp"
        android:textStyle="bold"
        android:fontFamily="@font/montserratmedium"
        android:textColor="@android:color/white"
        android:gravity="center"
        android:layout_marginTop="5dp"
        android:padding="16dp"
        android:elevation="8dp" />
    <!-- Main Content -->
    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp"
        app:layout_behavior="@string/appbar_scrolling_view_behavior">
```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center_horizontal">

    <!-- Total Votes Card -->
    <androidx.cardview.widget.CardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="16dp"
        android:elevation="10dp"
        android:background="@color/white"
        app:cardCornerRadius="16dp"
        android:layout_marginTop="60dp"
        android:padding="20dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center">

            <ImageView
                android:layout_width="48dp"
                android:layout_height="48dp"
                android:src="@drawable/ic_votes"
                app:tint="@android:color/holo_blue_dark"
                android:layout_marginEnd="12dp" />

            <LinearLayout
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Total Votes"
                    android:textSize="20sp"
                    android:textStyle="bold"
                    android:textColor="@color/black" />

                <TextView
                    android:id="@+id/total_votes"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="0"
                    android:textSize="24sp"
                    android:textStyle="bold"
                    android:textColor="@android:color/holo_blue_dark" />

```

```

<TextView
    android:id="@+id/winner_name"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/winner_n_a"
    android:textSize="20sp"
    android:textStyle="bold"
    android:textColor="@android:color/holo_green_dark"
    android:layout_marginTop="10dp"/>
</LinearLayout>
</LinearLayout>
</androidx.cardview.widget.CardView>

<!-- Admin Options -->

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:elevation="8dp"
    android:background="@color/white"
    app:cardCornerRadius="16dp"
    android:padding="16dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:padding="10dp">
<ImageView
    android:layout_width="32dp"
    android:layout_height="50dp"
    android:src="@drawable/users_manage"
    app:tint="@android:color/holo_blue_dark"
    android:layout_marginEnd="8dp" />

    <Button
        android:id="@+id/manage_users"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Manage Users"
        android:backgroundTint="@android:color/holo_blue_dark"
        android:textColor="@android:color/white"
        android:textSize="16sp"
        android:padding="12dp" />
</LinearLayout>

```

```

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="8dp" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:padding="10dp">

    <ImageView
        android:layout_width="32dp"
        android:layout_height="32dp"
        android:src="@drawable/analysis"
        app:tint="@android:color/holo_orange_dark"
        android:layout_marginEnd="8dp" />

    <Button
        android:id="@+id/view_results"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="View Results"
        android:backgroundTint="@android:color/holo_orange_dark"
        android:textColor="@android:color/white"
        android:textSize="16sp"
        android:padding="12dp" />
</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@android:color/darker_gray"
    android:layout_marginVertical="8dp" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:padding="10dp">

    <ImageView
        android:layout_width="32dp"
        android:layout_height="32dp"
        android:src="@drawable/ic_logout"
        app:tint="@android:color/holo_red_dark"
        android:layout_marginEnd="8dp" />

```

```

<Button
    android:id="@+id/logout_button"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Logout"
    android:backgroundTint="@android:color/holo_red_dark"
    android:textColor="@android:color/white"
    android:textSize="16sp"
    android:padding="12dp" />
</LinearLayout>

</LinearLayout>
</androidx.cardview.widget.CardView>

</LinearLayout>
</androidx.core.widget.NestedScrollView>

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Java Code:

```

package com.example.evm;

import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;
public class AdminActivity extends AppCompatActivity {

    private TextView totalVotes, winnerName;
    private Button manageUsers, viewResults, logoutButton;
    private DatabaseReference databaseReference;
    private SharedPreferences sharedPreferences;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_admin);
    }
}

```

```

// Initialize UI components
totalVotes = findViewById(R.id.total_votes);
winnerName = findViewById(R.id.winner_name);
manageUsers = findViewById(R.id.manage_users);
viewResults = findViewById(R.id.view_results);
logoutButton = findViewById(R.id.logout_button);

// Firebase Database Reference
databaseReference = FirebaseDatabase.getInstance().getReference();

// Initialize SharedPreferences
sharedPreferences = getSharedPreferences("loginPrefs", MODE_PRIVATE);

// Fetch and update total votes and winner
fetchTotalVotes();

// Manage Users button click event
manageUsers.setOnClickListener(v -> {
    Intent intent = new Intent(AdminActivity.this, ManageUsersActivity.class);
    startActivity(intent);
});

// View Results button click event
viewResults.setOnClickListener(v -> {
    Intent intent = new Intent(AdminActivity.this, ElectionResultsActivity.class);
    startActivity(intent);
});

// Logout button click event
logoutButton.setOnClickListener(v -> logoutAdmin());
}

// Fetch total votes and winner name from Firebase
private void fetchTotalVotes() {
    databaseReference.child("electionResults").addListenerForSingleValueEvent(new
ValueEventListener() {
        @Override
        public void onDataChange(@NonNull DataSnapshot snapshot) {
            if (snapshot.exists()) {
                for (DataSnapshot electionSnapshot : snapshot.getChildren()) {
                    Long totalVotesValue = electionSnapshot.child("TotalVotes").getValue(Long.class);
                    String winner = electionSnapshot.child("Winner").getValue(String.class);
                }
            }
        }
    });
}

// Correctly display total votes from electionResults
if (totalVotesValue != null) {
    totalVotes.setText("Total Votes: " + totalVotesValue);
} else {
    totalVotes.setText("Total Votes: 0");
}

```



```

// Correctly display winner name
    if (winner != null) {
        winnerName.setText("Winner: " + winner);
    } else {
        winnerName.setText("Winner: N/A");
    }
}
}

@Override
public void onCancelled(@NonNull DatabaseError error) {
    Toast.makeText(AdminActivity.this, "Failed to load votes", Toast.LENGTH_SHORT).show();
}
});
}

// Logout admin and redirect to LoginActivity
private void logoutAdmin() {
    sharedPreferences.edit().clear().apply(); // Clear login session
    Toast.makeText(AdminActivity.this, "Logged out successfully", Toast.LENGTH_SHORT).show();
    Intent intent = new Intent(AdminActivity.this, Login.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    startActivity(intent);
    finish();
}
}

```

APPENDIX B

Project Timeline

TITLE	“Digital Electoral System”	
MEETING NO.	DESCRIPTION OF MILESTONE	DURATION
1	Finalization of project idea and selection of supervisor	3 weeks
2	Submission of Project Proposal	1 week
3	Requirements gathering	3 weeks
4	Define the scope of the project.	2 weeks
5	Making ERD and implementation of model based on the ERD	4 weeks
6	Coding the front end of the application	6 weeks
7	Server-side coding of both front end and back end	5 weeks
8	Implementing Firebase Real-Time Database	4 weeks
9	Back end of application	6 weeks
10	Encryption	4 weeks
11	Testing phase 1	2 weeks
12	Final Documentation implementation	1 week
13	Final testing of project with hardware and software	2 weeks
14	FYP -II Demo Presentation	1 week

Members Signature

Supervisor Signature

APPENDIX C

SUPERVISOR MEETING LOG

MEETING NO.	Meeting Agenda	Date
1	Idea of FYP	18-12-2023
2	FYP Proposal	15-02-2024
3	FYP Proposal Defense	16-02-2024
4	Mid Documentation	26-4-2024
5	Front End UI Designing	14-5-2024
6	Server-Side Integration	21-6-2024
7	Mid Defense	8-7-2024
8	Digital Electoral System	05-08-2024
9	Backend Development	20-09-2024
10	Database Integration	08-10-2024
11	Result Compilation	17-11-2024
12	Diagrams	07-12-2024
13	Complete Documentation	15-02-2025
14	Testing	10-03-2025

Supervisor Signature